

Instituto Tecnológico Autónomo de México (ITAM)

Producto de datos para pronóstico de ventas en AWS

Reporte del POC - 1C Company

Avril Salazar Rodríguez

Héctor Vilchis Peralta

Arquitectura de Productos de Datos y Métodos de Gran Escala

Mayo 2026

Índice

1. Introducción	3
2. Problema de negocio y traducción a requisitos	3
3. Descripción general de la solución	5
4. Arquitectura de la solución	6
4.1. Componentes principales	7
5. Modelo de datos	8
5.1. Base operacional en Amazon RDS	11
6. Pipeline de datos y <i>machine learning</i>	12
6.1. Estrategia de inferencia	15
6.2. Artefactos de ModelOps consumidos por la aplicación	16
6.3. Política híbrida de predicción	16
6.4. Relación entre inferencia, evaluación y operación	17
7. Estrategia de inferencia en <i>Streamlit</i>	18
8. Estrategia de modelación	19
8.1. Modelos considerados	19
8.2. Motivación de la política híbrida	20
8.3. Funcionamiento del ruteo híbrido	21
8.4. Selección del modelo y trazabilidad	22
8.5. Control de <i>leakage</i>	22
8.6. Resultado de la estrategia	22
9. Evaluación del modelo	23
10. Tour de la aplicación	25
10.1. Resumen	26
10.2. Inferencia individual	26
10.3. Batch CFO	27
10.4. Batch por archivo cargado	29
10.5. Evaluación	30
10.6. KPIs	31
10.7. Feedback y productos problemáticos	32
10.8. Model Registry	34
11. Evidencias de AWS	34

12.Costos y operación	39
13.Operación, estabilidad y seguridad	41
14.Cobertura de la rúbrica	41
15.Limitaciones y próximos pasos	42
16.Uso de herramientas de IA	42
17.Bibliografía y recursos consultados	42

1 Introducción

En este proyecto se desarrolló un *Minimum Viable Product* (MVP) de un producto de datos para consultar pronósticos mensuales de ventas de 1C Company. El objetivo principal fue transformar un flujo de *machine learning* que originalmente vivía en *notebooks* y tareas técnicas en una aplicación web que pudiera ser utilizada por perfiles de negocio, sin necesidad de ejecutar código, abrir un ambiente de desarrollo o depender de una computadora local.

La solución se implementó como una aplicación de *Streamlit* desplegada en AWS mediante contenedores. La imagen de la aplicación se construyó con *Docker*, se almacenó en Amazon ECR y se ejecutó en Amazon ECS con *Fargate*, detrás de un *Application Load Balancer* que permite acceder a la app desde una URL pública. A través de esta aplicación, el usuario puede consultar predicciones, filtrar resultados por tienda, producto o categoría, generar archivos *batch* para el CFO, revisar métricas de evaluación del modelo, consultar el *Model Registry* y capturar observaciones del negocio sobre productos que requieren revisión.

Además, la aplicación se conecta con servicios persistentes como Amazon S3 y Amazon RDS. Amazon S3 se utiliza para almacenar artefactos de *ModelOps*, predicciones, archivos generados para el CFO y resultados de inferencia por archivo cargado. Amazon RDS, con PostgreSQL, se utiliza para guardar información operacional como historial de archivos generados, eventos de uso y *feedback* del negocio. Esta separación permite que la información importante no dependa del ciclo de vida del contenedor, ya que si la tarea de ECS se reemplaza, reinicia o despliega una nueva versión, los datos relevantes permanecen en servicios persistentes.

El proyecto también considera una ruta de evolución para actualizar los modelos cuando lleguen nuevos datos. Para ello, se preparó un flujo de *ModelOps* que separa el cómputo pesado de la interfaz de usuario. En lugar de entrenar, recalcular *features* o generar todo el *forecast* dentro de *Streamlit*, la arquitectura propuesta publica artefactos ya procesados en Amazon S3 bajo una carpeta de *latest*. La aplicación consume esos artefactos para mantenerse rápida y estable. En una versión productiva, los procesos de *feature engineering*, entrenamiento, evaluación y generación de predicciones podrían ejecutarse como procesos externos a la app, por ejemplo mediante *jobs* programados, Amazon SageMaker Processing o pipelines equivalentes. AWS Glue Data Catalog queda como una capa de metadatos para que los archivos en S3 puedan ser consultados posteriormente desde herramientas analíticas como Amazon Athena o soluciones de BI.

2 Problema de negocio y traducción a requisitos

La solución desarrollada consiste en una aplicación de *Streamlit* desplegada en AWS mediante ECS *Fargate*. Esta app funciona como la capa de consumo del producto de datos donde el usuario entra desde un navegador, consulta pronósticos, genera archivos, revisa métricas y captura observaciones. La aplicación no depende de una ejecución local, por lo que puede abrirse desde cualquier equipo mientras el servicio de ECS se mantenga activo. El diseño separa tres partes principales. La primera es la interfaz de usuario, donde se muestran las vistas de resumen, inferencia individual, batch CFO,

evaluación, KPIs y feedback. La segunda es la capa de persistencia, donde se guardan archivos y datos operacionales en S3 y RDS. La tercera es la capa de actualización de modelos, donde se dejó preparado un flujo ModelOps para procesar nuevos datos y generar predicciones actualizadas sin cargar ese trabajo a Streamlit. En la versión actual del MVP, se utilizan predicciones precomputadas para los casos de mayor tamaño, como el catálogo completo o todos los productos de una tienda. Esta decisión permite que la experiencia sea rápida para el usuario, ya que Streamlit no tiene que recalcular miles de predicciones en cada interacción. Para casos más pequeños, como inferencia individual o archivos CSV cargados por el usuario, la app sí utiliza el modelo serializado dentro del contenedor y genera la predicción en el momento. La infraestructura principal se desplegó con CloudFormation. Esto permite reconstruir la solución de forma más ordenada y evita depender de configuraciones manuales hechas desde la consola. La imagen de la aplicación se guarda en ECR y se ejecuta en ECS Fargate. La base de datos PostgreSQL vive en RDS y sus credenciales se consultan mediante Secrets Manager. Los archivos generados para el CFO se guardan en S3, mientras que Glue Data Catalog se utiliza para registrar tablas analíticas.

El siguiente cuadro resume lo anterior:

Cuadro 1: Traducción de la voz del cliente a requisitos del MVP.

Stakeholder	Necesidad	Respuesta en el MVP
Planeación de demanda	Entrar a una URL, filtrar por tienda, producto o categoría y ver pronósticos rápido.	Streamlit público en ECS Fargate, navegación optimizada y predicciones precomputadas para consultas grandes.
Finanzas / CFO	Generar archivos para una tienda, categoría o catálogo completo y guardarlos en un lugar persistente.	Vista Batch CFO con descarga CSV, guardado en S3 y registro en historial. Los CFO exports se particionan por tienda o categoría.
BI	Poder hacer cortes y reutilizar los pronósticos.	Archivos en S3, catálogo Glue y outputs ModelOps con tablas parquet/csv.
COO	La demo no debe depender de una laptop.	App desplegada como servicio persistente en ECS Fargate detrás de un ALB.
CFO	Costo mensual claro y controlado.	Estimación mensual de infraestructura, escenario estable y escenario optimizado.

Stakeholder	Necesidad	Respuesta en el MVP
On-call / plataforma	Logs, trazabilidad y respuesta ante fallas.	CloudWatch Logs, historial RDS/S3, eventos de uso y ajuste de ALB/Fargate para estabilidad.
CTO	Ruta para actualizar modelos con nuevos datos.	Flujo ModelOps con S3 como repositorio de artefactos, registry de modelos y outputs versionados.
Ciberseguridad	No guardar datos críticos dentro de la app.	S3, RDS y Secrets Manager como capas persistentes externas al contenedor.
Applied Scientist	Evaluar calidad y comunicar errores por grupo/producto.	Vista de Evaluación, KPIs, curvas de demanda, comparación contra naive y tablas por categoría/producto/tienda.
Planeación de inventarios	Capturar observaciones de negocio.	Vista Feedback con registro de comentarios, productos sobre-/subestimados y sugerencias de revisión.

3 Descripción general de la solución

La solución desarrollada consiste en una aplicación de *Streamlit* desplegada en AWS mediante Amazon ECS con *Fargate*. La aplicación funciona como la capa de consumo del producto de datos: el usuario entra desde un navegador, consulta pronósticos, genera archivos para negocio, revisa métricas de desempeño del modelo, analiza productos con errores o baja actividad y captura observaciones operativas. Al estar desplegada como servicio en AWS, la app no depende de una computadora local ni de que alguien ejecute un *notebook*; basta con que el servicio de ECS esté activo para que pueda ser consultada desde la URL pública.

El diseño final separa tres responsabilidades. La primera es la capa de interfaz, donde viven las vistas de Resumen, Inferencia individual, Batch CFO, Evaluación, KPIs, Feedback y Model Registry. La segunda es la capa de persistencia, donde Amazon S3 almacena artefactos, predicciones, archivos generados e historiales, mientras que Amazon RDS conserva información operacional como *feedback*, eventos de uso e historial de archivos. La tercera es la capa de *ModelOps*, que organiza los artefactos vigentes del modelo: predicciones, métricas, curvas de evaluación, *registry*, modelo *champion* y sugerencias de revisión. Esta separación permite que la interfaz se mantenga ligera y que el cómputo pesado no ocurra dentro de *Streamlit*.

En la versión final del MVP, los pronósticos de mayor tamaño se consumen desde archivos pre-

computados en S3. Esto aplica para consultas como catálogo completo, todos los productos de una tienda o un segmento/categoría completo. Esta decisión hace que la experiencia sea más rápida y estable, porque la app no recalcula miles de predicciones en cada interacción. Para casos puntuales, como inferencia individual o inferencia sobre un archivo CSV cargado por el usuario, la aplicación sí usa el modelo serializado dentro del contenedor. En esos casos, el resultado se genera en el momento y se acompaña de columnas de trazabilidad como `model_scope`, `routing_reason` y `decision_recommendation`.

También se incorporó un flujo de *batch* por archivo cargado. Este flujo permite subir un CSV con ID, `shop_id` e `item_id`, o bien un archivo con *features* completas. Cuando el archivo viene en formato tipo *test* o validación, la app completa las variables necesarias a partir de las *features* preparadas del dataset. Si el par tienda-producto no existe en la referencia, se trata como un caso *cold-start* y se aplica una política conservadora. Las predicciones generadas por archivos cargados se guardan en una carpeta exclusiva de S3, separada de los archivos CFO normales.

La vista Batch CFO quedó orientada a generar archivos de negocio. El usuario puede producir archivos para todos los productos de una tienda, para un segmento/categoría o para el catálogo completo. Estos archivos se guardan en S3 con una estructura clara: por `shop_id` cuando el alcance es una tienda, por `item_category_id` cuando el alcance es una categoría, y en una carpeta general cuando el alcance es catálogo completo. Además, la aplicación registra el historial de archivos generados para que el usuario pueda consultar cuántos registros se exportaron, cuál fue el total pronosticado y en qué ruta S3 quedó guardado el archivo.

La solución incluye varias vistas para explicar la calidad y el comportamiento del modelo. En Evaluación se comparan las curvas de predicción contra el valor real por rangos de demanda, y también se pueden comparar modelos como el Hurdle HGB, el LightGBM original, el modelo *naive* y otros candidatos. En KPIs se muestran errores por categoría, producto y tienda, además de una sección de productos con baja actividad reciente. En Feedback se presentan productos sobreestimados, subestimados y registros sugeridos para revisión, junto con una explicación de la razón por la que fueron marcados. El Model Registry conserva la comparación de modelos y permite identificar el modelo *champion*, sus métricas y su descripción.

La infraestructura principal se desplegó con AWS CloudFormation para mantener una definición reproducible de los recursos. La imagen de la aplicación se construye con *Docker*, se almacena en Amazon ECR y se ejecuta en Amazon ECS con *Fargate*. El acceso público se realiza mediante un *Application Load Balancer*. Los archivos y artefactos viven en Amazon S3; la base operacional vive en Amazon RDS con PostgreSQL; las credenciales se consultan con AWS Secrets Manager; los logs se revisan en Amazon CloudWatch; y AWS Glue Data Catalog se utiliza como capa de metadatos para las tablas analíticas construidas sobre archivos en S3. Con esta arquitectura, el MVP queda preparado para operar como una aplicación de datos real y no solo como un experimento local.

4 Arquitectura de la solución

La arquitectura se diseñó para mantener separadas tres responsabilidades: la capa de consumo, la capa de almacenamiento y la capa de ModelOps. La aplicación de Streamlit se encarga de la experiencia

del usuario. S3 y RDS se encargan de persistir resultados y datos operacionales. El flujo de ModelOps produce artefactos, predicciones, métricas y archivos de evaluación que la app consume sin recalcular todo en cada interacción.

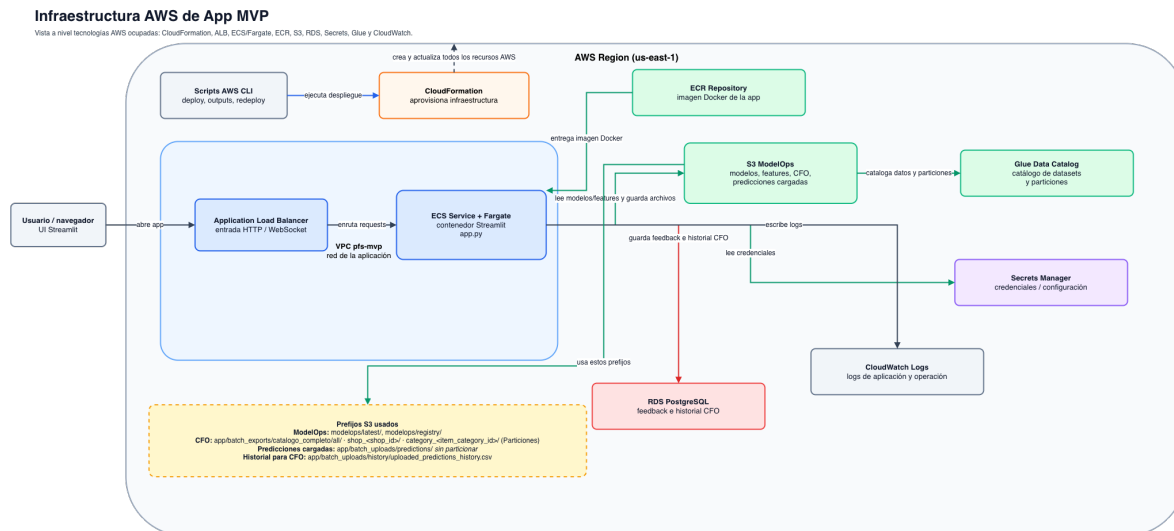


Figura 1: Diagrama de arquitectura general del MVP.

4.1 Componentes principales

- **Application Load Balancer.** Expone la aplicación mediante una URL pública. Se ajustó el idle timeout para reducir cortes de conexión en sesiones largas de Streamlit.
- **Amazon ECS Fargate.** Ejecuta la app como contenedor. No se administra EC2 directamente. Para estabilizar la app se subió la tarea a 2 vCPU y 4 GB durante la demo.
- **Amazon ECR.** Guarda la imagen Docker de la app. El pipeline de despliegue construye la imagen, la empuja a ECR y actualiza ECS.
- **Amazon S3.** Guarda salidas ModelOps, predicciones precomputadas, archivos CFO, predicciones de archivos cargados, métricas, curvas y artefactos del flujo de datos.
- **AWS Glue Data Catalog.** Registra tablas analíticas sobre las salidas en S3. Permite que el producto no se limite a la UI y pueda ser consumido por herramientas de análisis.
- **Amazon RDS PostgreSQL.** Guarda datos operacionales: feedback, historial CFO, eventos de uso y tablas de apoyo. No se usa para almacenar todo el histórico de ventas.
- **AWS Secrets Manager.** Almacena credenciales de RDS. La app no tiene usuario ni contraseña escritos en el código.
- **AWS CloudFormation.** Se utilizó para desplegar la infraestructura donde se usaron plantillas de CloudFormation para aprovisionar recursos persistentes y de despliegue como el *bucket* de Amazon S3, la base Amazon RDS con PostgreSQL, roles IAM, permisos, el servicio de Amazon ECS con *Fargate* y el *Application Load Balancer* que expone la aplicación.
- **Amazon CloudWatch Logs.** Centraliza logs del contenedor, útil para depurar errores de

despliegue, reinicios, permisos y fallas de conexión.

No se usó SageMaker Batch Transform como camino principal (aunque sí se consideró como alternativa para ejecutar inferencias batch) debido a que lo pensamos más práctico, para este MVP con bajo volumen de tráfico y necesidad de respuesta inmediata, precomputar predicciones grandes y ejecutar inferencias pequeñas dentro de Streamlit era más simple, barato y estable. SageMaker Batch Transform queda como alternativa para producción si el volumen crece o si los jobs batch se vuelven demasiado grandes para el contenedor, debido a que ya teníamos *scripts* pensados para orquestar *jobs* en AWS, por ejemplo con un SageMaker Execution Role con instancia tipo ml.m5.large, su intención era que SageMaker pudiera tomar datos desde S3, ejecutar entrenamiento o calificar datos y guardar artefactos o predicciones de regreso en S3 cada cierto tiempo o con un *trigger* accionable cuando detectara nueva información.

5 Modelo de datos

El modelo de datos del proyecto se dividió en dos capas. Por un lado, Amazon S3 y AWS Glue Data Catalog funcionan como la capa analítica principal: ahí viven los archivos *Parquet*, las predicciones, las métricas de evaluación y los artefactos de *ModelOps*. Por otro lado, Amazon RDS se usa como base operacional para guardar información generada por la aplicación, como *feedback*, eventos de uso e historial de archivos.

El diagrama entidad-relación principal corresponde al Glue Catalog, no al RDS. Este diagrama resume las tablas analíticas que consume la aplicación para mostrar pronósticos, métricas, comparaciones de modelos y explicabilidad del *routing*.

Diagrama Entidad Relación de los datos consumidos en la APP.

Tablas principales del Glue Catalog, ModelOps, batch CFO y carga batch del MVP.

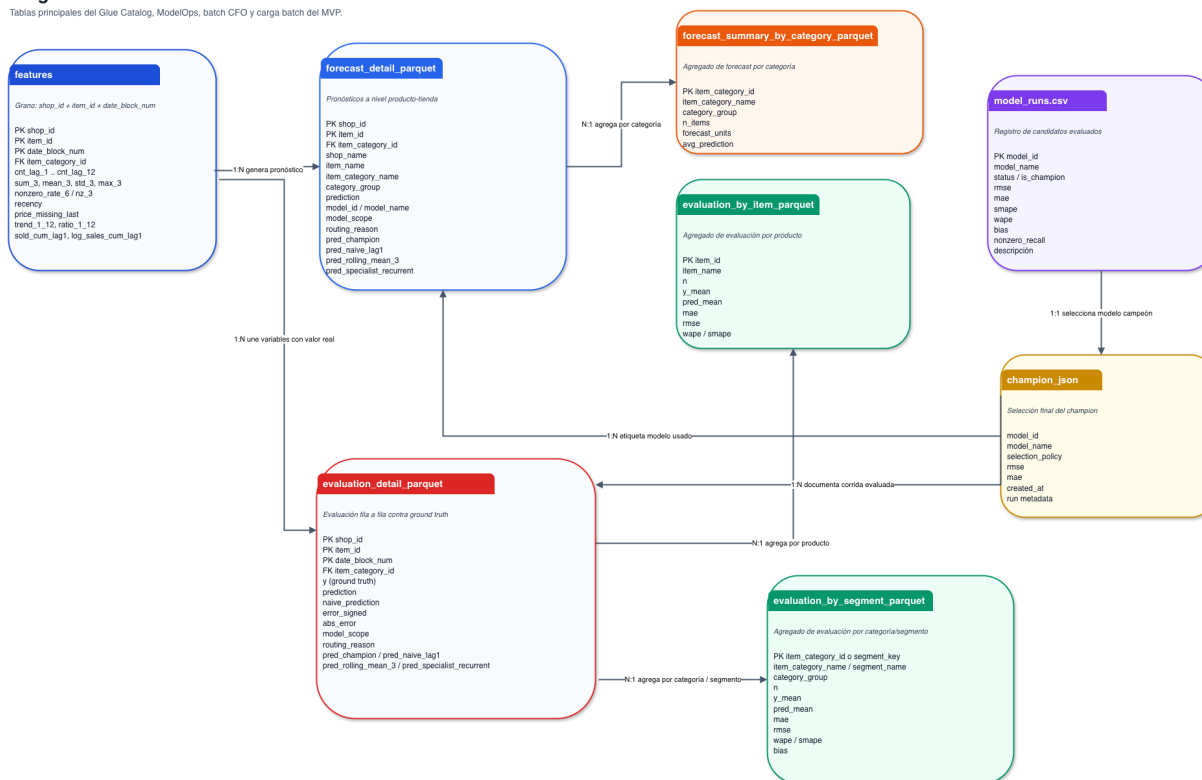


Figura 2: Diagrama entidad-relación principal del Glue Catalog con tablas de *features*, pronósticos, evaluación y registro de modelos.

Cuadro 2: Tablas principales consideradas en el Glue Catalog.

Tabla / artefacto	Propósito	Nivel de detalle y uso
features	Contiene las variables explicativas utilizadas para generar las predicciones. Incluye rezagos de ventas, medias móviles, frecuencia de ventas, recencia, variables de precio y señales acumuladas.	Se construye a nivel tienda, producto y mes. Es la entrada principal para los procesos de <i>scoring</i> , inferencia individual e inferencia por archivo cargado.
forecast_detail	Guarda la predicción final por producto y tienda, junto con la explicación de qué modelo o política produjo el resultado.	Se usa como tabla base de pronósticos. Incluye predicción final, modelo utilizado, alcance del modelo, razón de ruteo y predicciones candidatas como <i>naive</i> , promedio móvil o especialista recurrente.

Tabla / artefacto	Propósito	Nivel de detalle y uso
<code>evaluation_detail</code>	Guarda la evaluación fila a fila contra el valor real observado. Permite calcular errores, comparar contra el modelo <i>naive</i> y revisar casos de sobreestimación o subestimación.	Se usa para construir las métricas de validación. Contiene el valor real, la predicción del modelo, la predicción <i>naive</i> , el error firmado, el error absoluto y la política usada para pronosticar.
<code>forecast_summary_by_category</code>	Resume los pronósticos por categoría de producto. Permite identificar qué categorías concentran mayor volumen esperado.	Se usa en la pestaña de resumen para mostrar las categorías con mayor pronóstico total y apoyar la lectura agregada del catálogo.
<code>evaluation_by_item</code>	Resume el desempeño por producto. Ayuda a detectar productos con errores altos o comportamiento atípico.	Se usa en las vistas de evaluación, KPIs y feedback. Presenta métricas como promedio real, promedio predicho, MAE, RMSE, WAPE y SMAPE.
<code>evaluation_by_segment</code>	Resume el desempeño por categoría o segmento. Permite encontrar grupos donde el modelo falla más o donde una política específica tiene mejor desempeño.	Se usa para comparar desempeño entre categorías. Incluye nombre de categoría, grupo, promedio real, promedio predicho, error, sesgo y métricas de demanda no cero.
<code>model_runs</code>	Registra los modelos candidatos evaluados. Funciona como un <i>Model Registry</i> ligero para comparar modelos, políticas y <i>baselines</i> .	Se usa en la pestaña de <i>Model Registry</i> . Contiene nombre del modelo, estatus, marca de <i>champion</i> , RMSE, MAE, SMAPE, WAPE, sesgo y descripción del modelo.
<code>champion</code>	Indica cuál es el modelo seleccionado como modelo vigente. Resume la política de selección y las métricas principales del modelo elegido.	Se usa para mostrar el modelo en uso dentro de la aplicación y para mantener trazabilidad de la versión activa del sistema de pronóstico.

La relación central del modelo analítico es la siguiente: la tabla **features** alimenta los procesos de inferencia y evaluación. A partir de esas variables se genera **forecast_detail**, que contiene la predicción final y la explicación de qué modelo o política se utilizó. Cuando existe *ground truth*, el resultado se compara en **evaluation_detail**. Después, esta tabla se agrega en dos niveles: por producto en **evaluation_by_item** y por categoría o segmento en **evaluation_by_segment**.

El *Model Registry* se representa mediante `model_runs.csv` y `champion.json`. El primero conserva

todos los candidatos evaluados, como el modelo *Hurdle HGB*, el *LightGBM* original de dos etapas, el *naive*, el promedio móvil, el especialista de demanda recurrente y el *hybrid router*. El segundo indica cuál fue el modelo seleccionado como *champion* según la política de selección definida, principalmente RMSE y MAE.

5.1 Base operacional en Amazon RDS

Amazon RDS se utiliza como la base operacional de la aplicación. No funciona como *data lake* ni como repositorio principal de archivos, este rol queda más bien en Amazon S3 y en el Glue Catalog. Su papel es conservar información generada durante el uso de la app: comentarios del negocio, historial de archivos, eventos operativos y metadatos que conviene consultar de forma transaccional.

Esta separación es importante porque el contenedor de ECS es reemplazable. Si la tarea se reinicia, se actualiza la imagen o se despliega una nueva versión, la información operativa permanece en RDS y los archivos permanecen en S3. En ese sentido, RDS complementa al Glue Catalog: Glue describe y organiza datos analíticos en S3, mientras que RDS conserva el estado operacional de la aplicación.

Cuadro 3: Tablas operacionales consideradas en Amazon RDS.

Tabla	Propósito	Uso dentro de la app
<code>business_feedback</code>	Guarda observaciones escritas por usuarios de negocio sobre productos, tiendas o categorías que requieren revisión.	Se captura desde la pestaña de <i>Feedback</i> . La información puede ser revisada por planeación, inventarios y el equipo de ML.
<code>problem_products</code>	Conserva productos o pares tienda-producto sugeridos para revisión por error alto, sobreestimación, subestimación, baja actividad o posible inactividad.	Alimenta la vista de <i>Feedback</i> y permite priorizar casos donde el modelo o la operación requieren atención.
<code>batch_exports</code>	Registra los archivos CFO generados desde la aplicación. Incluye alcance, número de registros, suma de predicciones y ruta en Amazon S3.	Se consulta en la pestaña <i>Batch CFO</i> para mostrar el historial de archivos descargados o guardados.
<code>app_usage_events</code>	Guarda eventos de uso de la aplicación, como inferencias individuales, generación de archivos, errores o acciones relevantes.	Sirve para monitoreo operativo y trazabilidad básica de lo que ocurre dentro de la app.
<code>model_metadata</code>	Guarda metadatos de modelos cuando se requiere persistir información fuera de S3, por ejemplo versión activa, métricas principales o fecha de publicación.	Complementa la pestaña de <i>Model Registry</i> . La fuente principal del registry sigue siendo S3, pero RDS puede conservar metadatos operativos.

Además de RDS, el flujo de archivos queda organizado en Amazon S3. Esta estructura separa archivos de negocio, predicciones cargadas por usuarios y artefactos de *ModelOps*:

- `app/batch_exports/`: almacena los archivos CFO normales. Estos archivos se guardan por alcance: tienda, categoría o catálogo completo.
- `app/batch_exports/todos_los_productos_de_una_tienda/shop_<id>/`: carpeta para archivos CFO generados para una tienda específica.
- `app/batch_exports/segmento_categoria/category_<id>/`: carpeta para archivos CFO generados para una categoría o segmento específico.
- `app/batch_exports/catalogo_completo/all/`: carpeta para archivos CFO de catálogo completo.
- `app/batch_uploads/predictions/`: almacena las predicciones generadas a partir de archivos CSV cargados por el usuario. En este caso se guarda un archivo por corrida, sin particionar por categoría o tienda.
- `app/batch_uploads/history/uploaded_predictions_history.csv`: historial exclusivo de predicciones generadas por archivo cargado.
- `modelops/latest/`: contiene los artefactos vigentes del flujo de *ModelOps*, como predicciones, métricas, curvas de evaluación, *registry* y sugerencias de revisión.

6 Pipeline de datos y *machine learning*

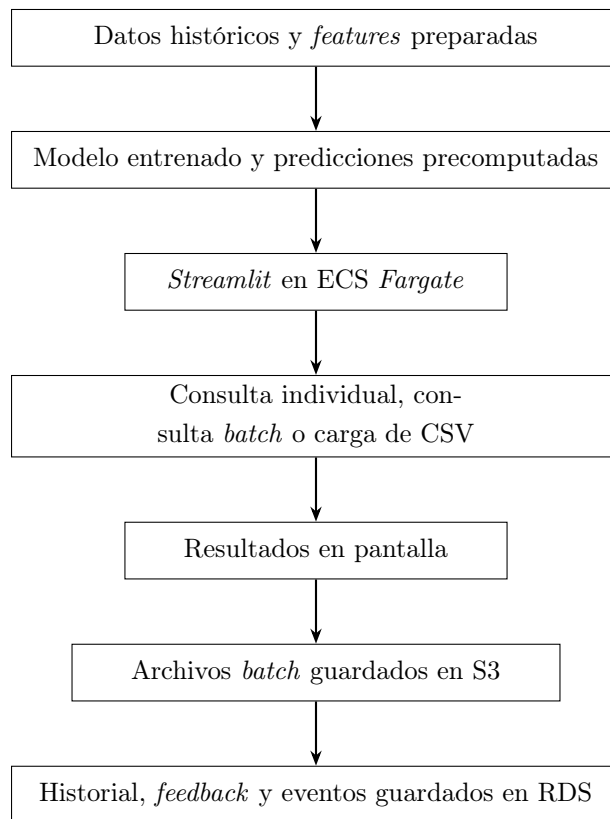
El flujo de datos del MVP parte de los datos históricos de ventas de 1C Company y de los artefactos construidos en tareas anteriores del curso. A partir de esos datos se generan *features*, se entrena el modelo, se producen predicciones y se preparan tablas para ser consumidas por la aplicación.

En la versión desplegada del MVP se tomó una decisión práctica: separar los casos grandes de los casos pequeños de inferencia. Para los casos grandes, como consultar todo el catálogo o todos los productos de una tienda, la app utiliza predicciones precomputadas. Esto evita que *Streamlit* tenga que ejecutar un proceso pesado cada vez que el usuario cambia un filtro. En cambio, la app solo lee resultados ya preparados y los presenta en pantalla.

Para los casos pequeños, como inferencia individual o predicción sobre un archivo CSV cargado por el usuario, la app carga el modelo serializado dentro del contenedor. Este modelo se guarda como `artifacts/model.joblib`.

El modelo se carga en *Streamlit* usando caché de recurso, de forma que no se vuelve a leer desde disco en cada interacción. Esto permite que la inferencia individual sea rápida y que el usuario pueda probar combinaciones tienda-producto sin esperar un *job* externo.

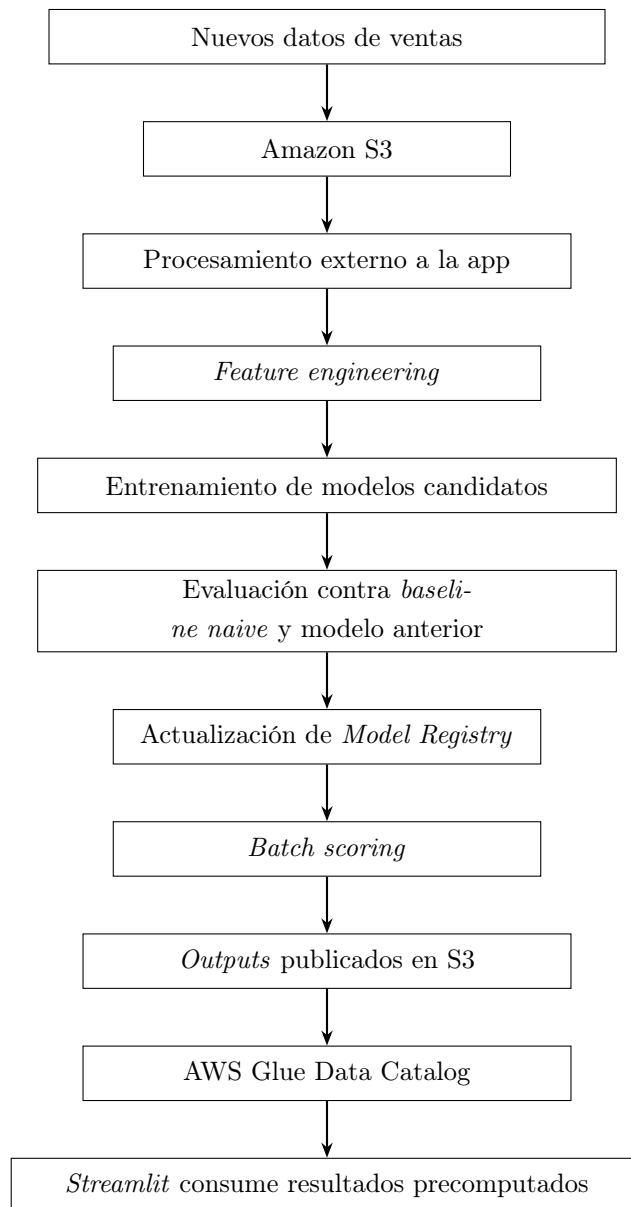
El flujo general de la aplicación se resume así:



Para los archivos del CFO, la aplicación filtra el conjunto de predicciones según el alcance seleccionado. Si el usuario elige una tienda, se generan predicciones para todos los productos de esa tienda. Si elige una categoría o segmento, se genera el archivo correspondiente a ese grupo de productos. Si selecciona catálogo completo, se genera el archivo completo. Posteriormente, el archivo se descarga desde la UI y también puede guardarse en S3 para que exista una copia persistente. La ruta del archivo y la metadata del proceso se registran en RDS.

Además del flujo actual, se dejó preparado un flujo de *ModelOps* para responder a la solicitud de cómo se actualiza el sistema cuando llegan nuevos datos. Este flujo separa el procesamiento pesado de la interfaz de usuario. La idea es que nuevos datos se suban a S3, después se ejecuten procesos de *feature engineering*, entrenamiento, evaluación y *batch scoring* fuera de *Streamlit*, y finalmente se publiquen nuevos *outputs* que la app pueda consumir.

El flujo de *ModelOps* propuesto queda representado de la siguiente forma:



Este flujo no quedó como un proceso automatizado en producción durante el MVP, sino como la ruta técnica preparada para evolucionar el sistema. La idea es que, cuando lleguen nuevos datos de ventas a Amazon S3 o se programe una actualización periódica, el procesamiento pesado ocurra fuera de la aplicación. En esa etapa se recalculan *features*, se entrenan o actualizan modelos candidatos, se comparan contra un *baseline naive* y contra el modelo vigente, y después se publican nuevos artefactos en una carpeta tipo `models/latest`. La aplicación no necesita reentrenar ni recalcular todo: únicamente vuelve a leer los artefactos vigentes.

En el MVP sí se dejó implementada la lógica de consumo de esos artefactos. La app lee predicciones, métricas, curvas de evaluación, sugerencias de revisión y el *Model Registry* desde S3. También se incorporó una política híbrida de ruteo que permite distinguir entre productos inactivos, casos donde conviene usar una señal *naive*, demanda recurrente y predicciones del modelo Hurdle HGB, por lo que de esta forma la aplicación ya está preparada para consumir nuevas versiones publicadas en S3 sin

cambiar la interfaz.

Como hemos comentado, nuestra estrategia evita que *Streamlit* se vuelva lenta o inestable por ejecutar procesos largos dentro de la sesión del usuario. La interfaz queda como capa de consulta, explicación y operación, mientras que el entrenamiento, evaluación y *batch scoring* pueden ejecutarse como procesos externos. En una versión productiva, esos procesos podrían programarse semanalmente o activarse cuando se detecten nuevos archivos en S3. SageMaker se consideró para esta fase como alternativa administrada para procesamiento o *batch scoring*, pero no quedó conectado al flujo final del MVP.

El manejo de credenciales se realiza mediante AWS Secrets Manager. La aplicación no guarda directamente el usuario ni la contraseña de RDS en el código; en su lugar, obtiene el secreto en tiempo de ejecución y construye la conexión a PostgreSQL. Esto mantiene separadas las credenciales de la lógica de la app y facilita cambiar la contraseña o rotar el secreto sin modificar el código del contenedor.

6.1 Estrategia de inferencia

La aplicación no ejecuta todos los pronósticos de la misma forma. Para mantenerla rápida y estable, se separaron los casos de inferencia según su tamaño y su uso dentro del producto de datos. Esta decisión fue importante porque no es lo mismo consultar un producto específico que generar un archivo para todo el catálogo.

Cuadro 4: Caminos de inferencia implementados en el MVP.

Camino de inferencia	Uso dentro de la aplicación	Decisión técnica
Predicciones precomputadas	Se usan para consultas grandes, como catálogo completo, todos los productos de una tienda o cortes por categoría.	La aplicación lee archivos ya generados en S3. Esto evita recalcular miles de predicciones dentro de <i>Streamlit</i> .
Inferencia individual	Se usa cuando el usuario consulta una combinación específica de tienda y producto.	El modelo serializado se carga dentro del contenedor desde <code>artifacts/model.joblib</code> . Se usa caché de recurso para no leerlo en cada interacción.
Batch por archivo cargado	Se usa cuando el usuario carga un CSV para probar inferencia sobre un conjunto nuevo de pares tienda-producto.	Si el archivo trae ID, <code>shop_id</code> e <code>item_id</code> , la app completa las variables usando las <i>features</i> preparadas. Si un par no existe en la referencia, se trata como caso <i>cold-start</i> .

Con esta estrategia, la aplicación conserva una buena experiencia de usuario. Las consultas grandes se atienden con artefactos precomputados y las consultas pequeñas se resuelven directamente en el

contenedor. Además, el flujo de archivo cargado permite probar escenarios nuevos sin modificar el pipeline principal ni reentrenar el modelo.

6.2 Artefactos de ModelOps consumidos por la aplicación

El flujo de *ModelOps* publica en S3 los archivos que la aplicación necesita para operar. Estos artefactos funcionan como la interfaz entre el procesamiento pesado y la capa de consumo. La app no necesita saber cómo se entrenó cada modelo en tiempo real; únicamente lee los resultados publicados y los presenta de forma organizada.

Cuadro 5: Artefactos principales de *ModelOps* consumidos por la aplicación.

Artefacto	Contenido	Uso en la app
<code>forecast_detail.parquet</code>	Predicciones por tienda-producto, modelo utilizado, <code>model_scope</code> , razón de ruteo y predicciones candidatas.	Resumen, Batch CFO, KPIs.
<code>evaluation_detail.parquet</code>	Valor real, predicción, predicción <i>naive</i> , error firmado, error absoluto y variables de evaluación.	Evaluación y análisis de errores.
<code>evaluation_curves_by_model.parquet</code>	Promedio real y promedio predicho por rango de demanda para distintos modelos.	Gráficas comparativas de modelos.
<code>model_metrics.json</code>	Métricas globales del modelo vigente y comparación con <i>baselines</i> .	Resumen ModelOps.
<code>model_runs.csv</code>	Historial de modelos candidatos, métricas, estatus y descripciones.	<i>Model Registry</i> .
<code>review_suggestions.parquet</code>	Registros sugeridos para revisión por error, sobreestimación, subestimación o baja actividad.	Feedback y revisión operativa.

Esto permite reemplazar los artefactos publicados sin cambiar la interfaz. Si en una siguiente corrida se genera un nuevo *champion*, nuevas métricas o nuevas predicciones, la aplicación puede consumir esos archivos desde la carpeta vigente de S3.

6.3 Política híbrida de predicción

Durante la evaluación se observó que un único modelo global no se comporta igual para todos los productos. En particular, los productos con ventas recurrentes, los productos casi inactivos y los productos con muchos ceros requieren tratamientos distintos. Por eso se incorporó una política híbrida que decide qué componente usar antes de generar la predicción final.

La política utiliza señales históricas disponibles al momento de predecir, como rezagos, recencia, medias móviles y frecuencia de ventas distintas de cero. No utiliza el valor real futuro. La decisión queda registrada en dos columnas: `model_scope`, que indica qué componente se usó, y `routing_reason`, que explica por qué se eligió ese componente.

Cuadro 6: Componentes de la política híbrida de predicción.

Componente	Regla de uso	Interpretación
<code>inactive:no_recent_sales</code>	Recencia alta o ventas recientes iguales a cero.	El par tienda-producto parece inactivo. La predicción se vuelve conservadora.
<code>baseline:naive_recent_demand</code>	Existe venta reciente positiva, por ejemplo en el último rezago.	La señal más reciente puede ser más informativa que un modelo global.
<code>specialist:recurrent_demand</code>	Las medias móviles o la frecuencia de ventas no cero indican recurrencia.	Se usa el componente especialista para demanda recurrente. Si no existe un artefacto especialista, se usa una aproximación transparente basada en señal reciente.
<code>challenger:hurdle_hgb</code>	No hay una señal clara de inactividad, recurrencia fuerte o naive reciente.	Se usa el modelo Hurdle HGB, que está diseñado para demanda dispersa y muchos ceros.

Esta política no reemplaza al *Model Registry*. El *registry* conserva todos los modelos evaluados y sus métricas, mientras que el ruteo híbrido explica qué componente se usó para cada predicción. Esto es útil en las vistas de Batch CFO, KPIs y Feedback, porque permite entender si una predicción vino de una regla de inactividad, de una señal *naive*, de un especialista o del modelo Hurdle HGB.

6.4 Relación entre inferencia, evaluación y operación

El flujo final conecta tres necesidades: generar predicciones, medir su calidad y dejar trazabilidad operativa. Las predicciones se consumen desde S3 o se generan dentro de la app cuando el caso es pequeño. La evaluación compara esas predicciones contra el valor real cuando este existe. La operación guarda archivos, historial y observaciones para que el negocio pueda revisar los resultados.

Cuadro 7: Relación entre las capas funcionales del MVP.

Capa	Qué resuelve	Evidencia en la app
Inferencia	Produce pronósticos para consultas individuales, cortes batch o archivos cargados.	Pestañas Inferencia individual y Batch CFO.
Evaluación	Mide el desempeño del modelo contra el valor real y contra <i>baselines</i> .	Pestaña Evaluación y curvas por rango de demanda.
Monitoreo de negocio	Identifica productos con error alto, baja actividad, sobreestimación o subestimación.	Pestañas KPIs y Feedback.
Persistencia	Guarda archivos generados, historial, feedback y rutas S3.	Amazon S3 y Amazon RDS.
Trazabilidad de modelos	Conserva modelos candidatos, métricas y modelo vigente.	Pestaña <i>Model Registry</i> .

Por ende, la app no se limita a mostrar una predicción, también explica qué política o modelo podría utilizar, permite evaluar su calidad, guarda los archivos generados y deja un canal para que usuarios de negocio indiquen qué productos requieren revisión o un mejor ajuste.

7 Estrategia de inferencia en *Streamlit*

La estrategia de inferencia se diseñó pensando en la experiencia del usuario final. El objetivo era que una persona de negocio pudiera entrar a la aplicación, filtrar información y obtener respuestas en segundos, sin esperar a que se ejecutara un proceso largo de entrenamiento o *scoring*. Por esta razón, cuando el usuario consulta el catálogo completo, todos los productos de una tienda o un segmento/categoría, la aplicación utiliza predicciones precomputadas. Es decir, *Streamlit* no recalcula todas las predicciones desde cero; lee un conjunto de resultados ya preparado y únicamente aplica los filtros necesarios para mostrar la información solicitada.

Esta decisión tiene tres ventajas principales. Primero, reduce la latencia de la interfaz, porque el usuario no espera a que se ejecute un *job* pesado. Segundo, reduce el costo operativo, porque no se dispara infraestructura adicional en cada clic. Tercero, hace más estable la aplicación, ya que los procesos *batch* grandes no dependen de que *Streamlit* mantenga viva una ejecución larga dentro de la sesión del usuario.

Para inferencia individual, la aplicación sí carga el modelo dentro del contenedor. Esto permite seleccionar una tienda y un producto, construir la entrada correspondiente y obtener una predicción en el momento. El modelo se carga con caché de recurso, por lo que no se vuelve a leer desde disco en cada interacción. Esto hace que la consulta individual sea suficientemente rápida para el MVP.

Además, se agregó un flujo de inferencia por archivo cargado. El usuario puede subir un CSV con las columnas completas de *features*, o bien un archivo tipo *test* con *ID*, *shop_id* e *item_id*. Cuando el archivo no trae todas las variables del modelo, la aplicación intenta completarlas a partir de las *features* preparadas del dataset. Si el par tienda-producto no existe en la referencia, se trata como un caso *cold-start* y se aplica una política conservadora. Este flujo demuestra que la aplicación no solo muestra resultados estáticos, sino que también puede generar predicciones nuevas a partir de un *input* cargado por el usuario.

Por otro lado, no se utilizó SageMaker Batch Transform como mecanismo principal desde la app pública. Se revisó como alternativa para ejecutar *scoring batch* administrado, pero no quedó conectado al flujo final del MVP. Para esta entrega, el foco fue ofrecer una experiencia estable y rápida. Ejecutar Batch Transform desde la interfaz podría ser útil en una versión posterior, pero también introduce tiempos de espera, estados intermedios y manejo de *jobs* que no eran necesarios para demostrar el valor inicial del producto.

8 Estrategia de modelación

La estrategia de modelación se diseñó tomando en cuenta una característica central del problema: la demanda mensual por tienda y producto tiene muchos ceros y pocos casos con ventas altas. Esto implica que un modelo puede obtener buen desempeño global simplemente prediciendo valores pequeños, pero aun así fallar en productos con demanda recurrente o en casos donde el último periodo trae una señal fuerte. Por esa razón, el MVP no se construyó alrededor de un único modelo aislado, sino alrededor de un conjunto de modelos candidatos y una política híbrida de decisión.

Ya que, como comentaba el proyecto, el objetivo no fue ganar la competencia original de Kaggle, sino construir un sistema de predicción que pudiera servir en la práctica. Para ello se compararon modelos más complejos contra *baselines* simples, especialmente contra un modelo *naive*. Esta comparación era importante porque, para series de tiempo o datos con estructura temporal, un modelo productivo debe demostrar que aporta valor frente a reglas sencillas como usar la venta del último periodo o un promedio móvil.

8.1 Modelos considerados

Los modelos incluidos en el *Model Registry* cumplen funciones distintas. Algunos son modelos predictivos principales, otros funcionan como *baselines* y otros se usan como componentes especializados para ciertos patrones de demanda.

Cuadro 8: Modelos y componentes considerados en la estrategia de modelación.

Modelo / componente	Qué hace	Por qué se incluyó
<i>Hurdle HGB</i>	Modelo de dos etapas. Primero estima si habrá venta o no; después estima las unidades esperadas cuando sí existe venta.	Es adecuado para datos con muchos ceros, porque separa el problema de ocurrencia de venta del problema de magnitud de venta.
<i>LightGBM</i> original de dos etapas	Modelo original del proyecto. También separa la probabilidad de venta y la estimación de unidades, usando las transformaciones y tratamientos previamente desarrollados.	Se mantuvo como modelo <i>incumbent</i> . Sirve como referencia productiva y evita perder el trabajo original que ya tenía buen desempeño.
<i>HistGradientBoosting Poisson</i>	Modelo de <i>gradient boosting</i> con pérdida Poisson para conteos no negativos.	Se incluyó porque las ventas son conteos. Es una alternativa cuando se quiere evitar predicciones negativas y modelar unidades vendidas.
Especialista de demanda recurrente	Componente pensado para pares tienda-producto con señales históricas de venta repetida. Se apoya en variables como medias móviles y frecuencia de ventas no cero.	Se incluyó porque algunos productos no son puramente intermitentes: tienen demanda recurrente y pueden beneficiarse de un tratamiento distinto al modelo global.
<i>Naive</i> de último periodo	Predice usando la venta observada en el último periodo disponible.	Es el <i>baseline</i> mínimo para demostrar que el modelo aporta valor. Si un modelo no supera al <i>naive</i> , no hay suficiente justificación para usarlo.
Promedio móvil	Usa el promedio de los últimos rezagos disponibles, por ejemplo los últimos tres periodos.	Funciona como una regla simple para capturar demanda reciente sin entrenar un modelo complejo. También ayuda a interpretar si el modelo está subestimando demanda recurrente.
Promedio histórico por producto	Predice a partir del comportamiento promedio histórico del producto.	Se usa como <i>fallback</i> cuando la información reciente es limitada o cuando se requiere una referencia simple de largo plazo.

8.2 Motivación de la política híbrida

Durante la evaluación se observó que los modelos no fallaban igual en todos los rangos de demanda. En demanda baja o cercana a cero, un modelo tipo *hurdle* puede ser bastante útil porque aprende a

distinguir entre venta y no venta. Sin embargo, en productos con señales recientes fuertes, una regla *naive* puede estar más cerca del valor real que un modelo global. Algo similar ocurre con productos recurrentes: si el historial muestra ventas repetidas, conviene usar un componente que responda mejor a esa recurrencia.

Por eso se incorporó una política híbrida. La idea no es escoger manualmente un modelo después de ver el resultado real, sino usar señales históricas disponibles antes de la predicción para decidir qué componente tiene más sentido para cada fila. Esta decisión queda registrada en columnas de trazabilidad, principalmente `model_scope`, `routing_reason` y `decision_recommendation`. Así, el usuario no solo ve la predicción final, sino también la razón por la cual esa predicción salió de un componente específico.

8.3 Funcionamiento del ruteo híbrido

La política híbrida utiliza variables generadas en el proceso de *feature engineering*, como rezagos de ventas, medias móviles, frecuencia de ventas no cero y recencia. Con esas señales se decide qué ruta usar para cada par tienda-producto.

Cuadro 9: Rutas de decisión usadas por la política híbrida.

Ruta	Criterio general	Interpretación
<code>inactive:no_recent_sales</code>	Recencia muy alta o suma reciente de ventas igual a cero.	El producto-tienda parece inactivo. La predicción se vuelve conservadora, normalmente cercana a cero.
<code>baseline:naive_recent_demand</code>	Existe venta positiva en el último periodo disponible.	La señal más reciente puede ser más informativa que el modelo global, por lo que se usa una regla <i>naive</i> .
<code>specialist:recurrent_demand</code>	La media móvil o la frecuencia de ventas no cero sugieren demanda recurrente.	Se usa el componente especialista para productos con ventas repetidas o comportamiento más estable.
<code>challenger:hurdle_hgb</code>	No se activa una regla de inactividad, <i>naive</i> reciente o demanda recurrente clara.	Se usa el modelo Hurdle HGB como modelo principal para demanda intermitente o dispersa.

Esta lógica permite que el sistema sea más flexible que un único modelo global. Por ejemplo, si un producto no se ha vendido recientemente, el sistema no fuerza una predicción positiva solo porque el modelo global encuentre algún patrón débil. Si el producto tuvo venta en el último periodo, la política puede usar esa señal reciente. Si existe recurrencia, se puede usar el especialista. En los demás casos, el modelo Hurdle HGB funciona como modelo principal.

8.4 Selección del modelo y trazabilidad

La selección del modelo *champion* se hizo considerando principalmente RMSE, con MAE como métrica complementaria. El RMSE ayuda a penalizar errores grandes, mientras que el MAE permite leer el error promedio en unidades de venta. Además, se mantuvieron métricas como SMAPE, WAPE, sesgo y recuperación de demanda no cero para revisar comportamiento operativo.

El *Model Registry* no se limitó a guardar el modelo ganador. También conserva modelos candidatos y *baselines*, como el *naive*, el promedio móvil, el LightGBM original y el especialista recurrente. Esto es útil porque permite explicar por qué se eligió un modelo y contra qué alternativas fue comparado. En la app, esta información aparece en la pestaña *Model Registry*, junto con descripciones de cada modelo.

La trazabilidad por fila se conserva mediante columnas como:

- **model_id**: identifica el artefacto o familia del modelo.
- **model_scope**: indica qué componente o política produjo la predicción final.
- **routing_reason**: explica la regla histórica que justificó el ruteo.
- **decision_recommendation**: traduce la decisión técnica a una explicación más legible para negocio.

Estas columnas aparecen en los archivos generados, en la vista Batch CFO y en las tablas de revisión. Esto ayuda a que las predicciones no se vean como una caja negra.

8.5 Control de *leakage*

Un punto importante fue evitar *data leakage*. Las decisiones de ruteo y las variables del modelo se construyen con información disponible antes del periodo que se quiere predecir. En particular, los rezagos, medias móviles, recencia y frecuencias de venta se calculan a partir del historial pasado. No se usa el valor real futuro para decidir si una fila debe irse por *naive*, especialista, regla de inactividad o Hurdle HGB.

Cuando la app completa columnas para un archivo tipo *test* cargado por el usuario, utiliza las *features* preparadas del dataset. Esto es válido mientras esas *features* hayan sido construidas únicamente con información histórica anterior al mes objetivo. Si el usuario carga pares tienda-producto que no existen en la referencia, la app no inventa historial. En esos casos trata el registro como *cold-start*, asigna señales históricas neutras y aplica una política conservadora.

Esta separación es importante: completar *features* desde una tabla preparada no es *leakage* por sí mismo. Se vuelve *leakage* si esa tabla fue construida usando ventas reales del periodo que se intenta predecir. Por eso, en una versión productiva, el proceso de *feature engineering* debe ejecutarse siempre con corte temporal controlado y antes de publicar las predicciones en S3.

8.6 Resultado de la estrategia

La estrategia final combina tres ideas: modelos candidatos, *baselines* interpretables y una política híbrida de ruteo. Con esto, el MVP no solo entrega una predicción, sino también una explicación de

cómo se llegó a ella. La aplicación puede mostrar el modelo ganador, comparar contra modelos simples, revisar dónde falla por rango de demanda y marcar productos que requieren atención.

Esta aproximación es adecuada para el MVP porque equilibra desempeño, interpretabilidad y operación. No intenta resolver todos los casos con un solo modelo ni sobrecomplica la arquitectura con un sistema de entrenamiento dentro de la app. El procesamiento pesado queda fuera de *Streamlit*; la app consume artefactos y ejecuta inferencia ligera cuando el usuario lo necesita.

9 Evaluación del modelo

La evaluación del modelo se realizó sobre un conjunto de validación que sí cuenta con *y* real observado. Este punto es importante porque las métricas de desempeño solo pueden calcularse cuando existe *ground truth*. Por ello, la pestaña de Evaluación de la aplicación se concentra en comparar predicciones contra valores reales históricos, mientras que las predicciones futuras o los archivos cargados por el usuario se tratan como *forecast* sin métrica de error hasta que exista el dato real.

La métrica principal de selección fue RMSE, porque penaliza con mayor fuerza los errores grandes. Esto es relevante en inventarios, donde una subestimación importante puede provocar subabasto y una sobreestimación importante puede generar exceso de inventario. MAE se utilizó como métrica complementaria porque permite leer el error promedio en unidades de venta de forma más directa. También se revisaron SMAPE, WAPE, sesgo y *recall* de ventas no cero para tener una lectura más completa del comportamiento del modelo.

La comparación contra un modelo *naive* fue central, ya que el modelo con mejores métricas debe justificar su uso al observar que mejora reglas simples como usar la venta del último periodo o un promedio móvil. Si un modelo no supera a estos *baselines*, su valor operativo es limitado, aunque sea más sofisticado. Por esa razón, el *Model Registry* conserva tanto modelos entrenados como reglas simples de comparación.

Cuadro 10: Modelos evaluados en el registry final. Los valores corresponden a la corrida mostrada en la app.

Modelo	RMSE	MAE	SMAPE	WAPE
Hurdle HGB	0.8842	0.2858	1.8852	1.1180
Especialista demanda recurrente	0.8849	0.3145	1.8924	1.2301
Hybrid Router: inactive + naive + specialist + HGB	0.9028	0.2723	0.6259	1.0653
Rolling mean últimos 3 lags	0.9491	0.2919	0.4908	1.1419
Naive último periodo	1.0843	0.3010	0.3249	1.1775

Aquí se puede observar que los modelos de dos etapas, en particular Hurdle HGB y el LightGBM original, se mantienen como los mejores candidatos por RMSE. Ambos superan al *naive* global en esta métrica, lo cual justifica conservar un modelo entrenado para el pronóstico principal. Sin embargo, la evaluación no se puede leer solamente con una métrica global. El conjunto de ventas tiene muchos ceros y pocos casos de demanda alta, como hemos ya comentado, por lo que una métrica agregada puede ocultar diferencias importantes entre rangos de demanda.

Por esta razón, la aplicación incluye curvas de calibración por rangos de demanda real. Estas gráficas comparan el promedio real contra el promedio predicho en distintos *buckets*. Ahí se observó que el modelo entrenado funciona bien en términos generales, pero que el *naive* puede ser competitivo en ciertos rangos, especialmente cuando hay señal reciente fuerte. Esta observación fue una de las razones para conservar *baselines* dentro del *Model Registry* y para incorporar la política híbrida de ruteo.

La evaluación también se desagrega por categoría, producto y tienda. Esto ayuda a pasar de una lectura general del modelo a una lectura accionable. Por ejemplo, si una categoría concentra errores altos, el equipo puede revisar si se trata de productos con demanda irregular, baja actividad, cambios de catálogo o señales recientes que el modelo no está capturando. De manera similar, los errores por producto permiten identificar casos concretos de sobreestimación o subestimación que pueden enviarse a revisión de negocio.

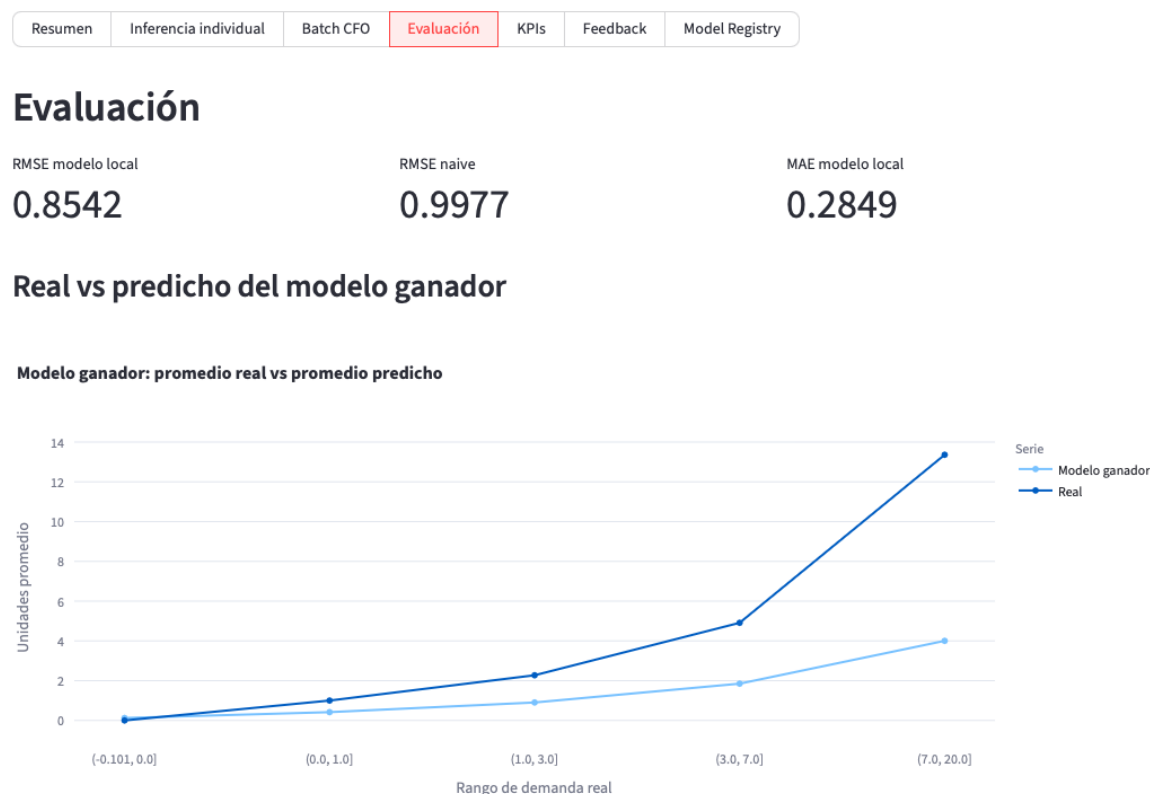


Figura 3: Evaluación: curva real contra modelos seleccionados por bucket de demanda.

Distribución de registros por rango de demanda real

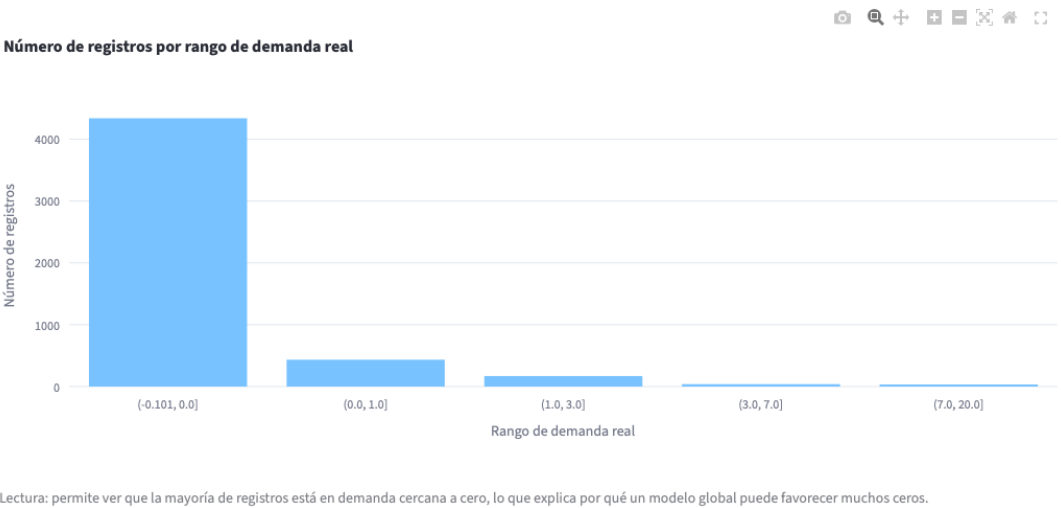
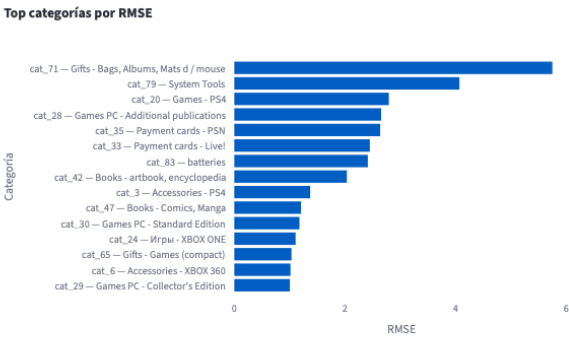
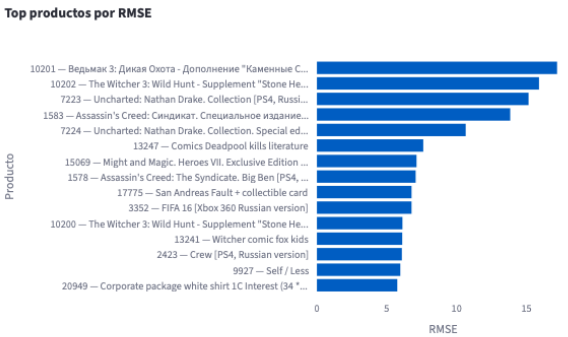


Figura 4: Distribución de registros por rango de demanda real.

Categorías con mayor RMSE



Productos con mayor RMSE



	item_category_id	item_category_name	category_group	n	y_mean	pred_
0	71	Gifts - Bags, Albums, Mats d / mouse	Gifts	42	18.2857	1
1	79	System Tools	System Tools	42	11.0476	5
2	20	Games - PS4	Games	5754	0.9741	0
3	28	Games PC - Additional publications	Games PC	2478	0.8434	0
4	35	Payment cards - PSN	Payment cards	168	3.9464	0
5	33	Payment cards - Live!	Payment cards	84	2.0119	0
6	83	batteries	batteries	168	2.0536	1
7	42	Books - artbook, encyclopedia	Books	504	1.2659	0

	item_category_id	item_category_name	category_group	item_id	item_name
0	28	Games PC - Additional publications	Games PC	10201	Ведьмак 3: Ди
1	20	Games - PS4	Games	10202	The Witcher 3: \
2	20	Games - PS4	Games	7223	Uncharted: Nat
3	20	Games - PS4	Games	1583	Assassin's Cree
4	20	Games - PS4	Games	7224	Uncharted: Nat
5	47	Books - Comics, Manga	Books	13247	Comics Deadpc
6	28	Games PC - Additional publications	Games PC	15069	Might and Magi
7	20	Games - PS4	Games	1578	Assassin's Cree

Figura 5: KPIs de RMSE por tienda, categoría y producto.

10 Tour de la aplicación

La aplicación quedó organizada en siete vistas. En esta sección se listan las capturas que deben actualizarse para el reporte final. Los nombres de archivo ya están codificados en LaTeX: basta con

guardar cada captura en la carpeta **figures/** con el nombre indicado.

10.1 Resumen

La vista de Resumen muestra el volumen total de predicciones, el modelo en uso, una muestra de forecast y gráficas de distribución, tiendas y categorías. También muestra la cobertura por `model\_scope`, útil para ver qué parte del catálogo fue ruteada a reglas de inactividad, naive, especialista o HGB.



Figura 6: Vista Resumen de la aplicación.

10.2 Inferencia individual

La inferencia individual permite consultar un par tienda-producto. El usuario puede buscar por ID o por nombre. La app muestra la predicción y contexto del producto, lo que cubre la necesidad de consulta puntual sin abrir notebooks.

Inferencia individual

Selecciona con la lista o escribe el ID manualmente.

Tienda	Producto
2 — Adygea TC "Mega" ▼	30 — 007: COORDINATES "SKAYFOLL" ▼
O escribe shop_id	O escribe item_id
Ej. 2	Ej. 2

Pronóstico próximo mes

0.07 unidades

Par seleccionado

	shop_id	item_id	shop_name	item_name	item_category_id	item_category_name	category_group
0	2	30	Adygea TC "Mega"	007: COORDINATES "SKAYFOLL"	40	Movie - DVD	Movie

> Ver features usadas

Figura 7: Vista de inferencia individual.

10.3 Batch CFO

Batch CFO permite generar archivos para todos los productos de una tienda, para una categoría o para el catálogo completo. Los archivos normales de CFO se guardan en S3 bajo `app/batch/_exports/`. Cuando el alcance es tienda, la ruta se particiona por `shop\_id`; cuando el alcance es categoría, se particiona por `item\_category\_id`. Esto facilita que Finanzas encuentre exactamente el archivo que pidió.

Batch CFO

> ¿Qué significa model_scope y routing_reason?

Alcance

☒ Todos los productos de una tienda

☐ Segmento / categoría

☐ Catálogo completo

Tienda

2 — Adygea TC "Mega"

O escribe shop_id

Ej. 2

Registros

Pronóstico total

Promedio

5,100

545.2

0.107

> Cómo leer model_id, model_scope, routing_reason y decisión final en Batch CFO

Vista previa del archivo CFO

	shop_id	shop_name	item_id	item_name	item_category_id	item_category_name	prediction	decision_recommendation	model_scope
1	2	Adygea TC "Mega"	31	007: COORDINATES "SKAYFOLL" (BD)	37	Movies - Blu-Ray	1	Baseline naive por demanda reciente	baseline:naive
2	2	Adygea TC "Mega"	32	1+1	40	Movie - DVD	0.2092	Modelo ganador Hurdle HGB	challenger:hurdle
3	2	Adygea TC "Mega"	33	1+1 (BD)	37	Movies - Blu-Ray	0.1548	Modelo especialista de demanda recurrente	specialist:recurrent
4	2	Adygea TC "Mega"	38	10 most popular comedy twentieth century 10DVD (rem)	41	Cinema - Collector	0	Regla cero / producto inactivo	inactive:no_recent_sales
5	2	Adygea TC "Mega"	42	100 Best romantic melodies (mp3-CD) (Digipack)	57	Music - MP3	0	Regla cero / producto inactivo	inactive:no_recent_sales
6	2	Adygea TC "Mega"	45	100 of the best folk songs (mp3-CD) (CD-Digipack)	57	Music - MP3	0	Regla cero / producto inactivo	inactive:no_recent_sales
7	2	Adygea TC "Mega"	51	100 best classical works (mp3-CD) (Digipack)	57	Music - MP3	0	Regla cero / producto inactivo	inactive:no_recent_sales

Figura 8: Batch CFO con selección de alcance y vista previa del archivo.

Qué significa cada routing/review reason en CFO

	review_reason	significado
0	Revisión por inactive:no_recent_sales / recency>=99_or_rolling_sum_6==0	Revisión por regla de inactividad o ausencia de ventas recientes.
1	Revisión por baseline:naive_recent_demand / cnt_lag_1>=1	Revisión por uso de señal reciente / baseline naive.
2	Revisión por challenger:hurdle_hgb / default_sparse_low_demand	Revisión por ruteo al modelo Hurdle para demanda baja/intermitente.
3	Revisión por specialist:recurrent_demand / rolling_mean_or_nonzero_rate_signal	Revisión por política de demanda recurrente o modelo especialista.

Generar archivo CFO y guardar en S3

Descargar archivo CFO

Historial de archivos generados

	export_id	scope	shop_id	records_count	total_prediction	s3_uri	status	created_at
0	16	Archivo cargado / predicción batch	None	5	1.2053	s3://pfs-modelops-494321812137-us-east-1/app/batch_uploads/predictions/upload/	generated	2026-05-02 19:37:18
1	15	Todos los productos de una tienda	36	5100	289.2244	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/todos_los_producto	generated	2026-05-02 19:32:36
2	14	Archivo cargado / predicción batch	None	25	0	s3://pfs-modelops-494321812137-us-east-1/app/batch_uploads/predictions/upload/	generated	2026-05-02 19:28:56
3	13	Segmento / categoría	None	84	100.4039	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/segmento__catego	generated	2026-05-02 15:56:20
4	12	Todos los productos de una tienda	12	5100	915.1263	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/todos_los_producto	generated	2026-05-02 08:29:47
5	11	Segmento / categoría	None	168	505.0936	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/segmento__catego	generated	2026-05-02 08:28:41
6	10	Segmento / categoría	None	42	729.1909	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/segmento__catego	generated	2026-05-02 07:04:42
7	9	Catálogo completo	None	214200	41870.1364	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/catálogo_completo/	generated	2026-05-01 23:41:25
8	8	Segmento / categoría	None	42	444.6147	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/segmento__catego	generated	2026-05-01 17:01:12
9	7	Todos los productos de una tienda	39	5100	755.8301	s3://pfs-modelops-494321812137-us-east-1/app/batch_exports/todos_los_producto	generated	2026-05-01 16:29:51

Figura 9: Batch CFO con guardado en S3 e historial de archivos generados.

10.4 Batch por archivo cargado

Dentro de Batch CFO se agregó un flujo de inferencia por archivo cargado. Este flujo acepta tres escenarios: archivo con features completas, archivo estilo test con ID, `shop\_id`, `item\_id`, o solicitud de catálogo completo. Si el par existe en el feature store preparado, la app completa las columnas; si no existe, lo trata como cold-start. Las predicciones cargadas se guardan en una carpeta única: `app/batch\_uploads/predictions/`. No se particionan por categoría porque se consideran corridas de predicción ad hoc.

Batch por archivo cargado

Puedes cargar un CSV con features completas o un CSV tipo test/validación con `ID`, `shop\_id`, `item\_id`. La app completará las features desde `data/prep/test_features.parquet` y pronosticará una fila por par producto-tienda.

> Columnas requeridas por el modelo

Subir CSV

batch_u...sample.csv
5.9KB

Archivo cargado: `batch_upload_mixed_catalog_sample.csv` – filas: 25, columnas: 47

Cómo interpretar el CSV cargado ⓘ

- ☒ Usar filas del CSV tal cual (requiere features completas)
- ☐ Completar/expandir usando features preparadas del dataset
- ☐ Pronosticar catálogo completo del dataset
- ☒ Aplicar política híbrida de ruteo a este archivo cargado ⓘ

Ejecutar inferencia sobre archivo cargado

Registros inferidos

25

Pronóstico total

1.67

Promedio

0.0669

Fuente de features usada: `uploaded_features_direct`

Estas predicciones usan la política híbrida: `inactive`, `naive reciente`, `especialista recurrente` o `HGB` según señales históricas.

Fuente del especialista recurrente: `rolling_mean_proxy: no_specialist_joblib`

Figura 10: Batch por archivo cargado con inferencia y política híbrida.

Guardar resultado de archivo cargado en S3

Resultado de inferencia cargada guardado en S3.

Ruta S3 exacta donde quedó guardado el resultado:

s3://pfs-modelops-494321812137-us-east-1/app/batch_uploads/predictions/uploaded_batch_predictions_20260503-171000_batch_upload_inactive_sample.csv

Historial exclusivo de predicciones cargadas actualizado en:

s3://pfs-modelops-494321812137-us-east-1/app/batch_uploads/history/uploaded_predictions_history.csv

Historial de predicciones por archivo cargado

Historial exclusivo de resultados generados desde la carga CSV de inferencia batch. No mezcla archivos CFO normales.

	created_at	source_file	partition_type	partition_label	segment_id	segment_name	item_category_id	item_category_name	category_group
0	2026-05-03T17:10:00+00:00	batch_upload_inactive_sample.csv	uploaded_prediction_file	sin_particiones	None	None	None	None	None
1	2026-05-02T19:37:18+00:00	batch_upload_exact_pairs.csv	uploaded_prediction_file	sin_particiones	None	None	None	None	None
2	2026-05-02T19:28:56+00:00	batch_upload_inactive_sample.csv	uploaded_prediction_file	sin_particiones	None	None	None	None	None
3	2026-05-02T19:06:05+00:00	batch_upload_mixed_catalog_sample.csv	uploaded_prediction_file	sin_particiones	None	None	None	None	None
4	2026-05-02T18:24:37+00:00	batch_upload_mixed_catalog_sample.csv	category	category_45	45	Books - Audioboo	45	Books - Audiobooks 1C	Books
5	2026-05-02T18:24:37+00:00	batch_upload_mixed_catalog_sample.csv	category	category_49	49	Books - Methodica	49	Books - Methodical ma	Books
6	2026-05-02T18:24:37+00:00	batch_upload_mixed_catalog_sample.csv	category	category_54	54	Books - Digital	54	Books - Digital	Books
7	2026-05-02T18:24:37+00:00	batch_upload_mixed_catalog_sample.csv	category	category_57	57	Music - MP3	57	Music - MP3	Music

Figura 11: Historial exclusivo de predicciones por archivo cargado.

10.5 Evaluación

La vista de Evaluación compara modelos contra ground truth. Incluye curva real contra modelos seleccionados, distribución de demanda real y tablas de performance. Esta vista es importante para explicar al negocio dónde el modelo acierta y dónde subestima.



Figura 12: Vista de Evaluación contra ground truth.

10.6 KPIs

La vista de KPIs muestra categorías, productos y tiendas con mayor RMSE. También incluye productos con baja actividad reciente, top tiendas con mayor número de productos de baja actividad y top categorías afectadas. Esto ayuda a separar errores de modelo de casos donde probablemente no tiene sentido abastecer.



Figura 13: Vista KPIs con errores y baja actividad reciente.

10.7 Feedback y productos problemáticos

La vista Feedback permite registrar observaciones en texto y revisar productos sobreestimados, subestimados o sugeridos para revisión. Las tablas muestran columnas de decisión como `decision\_recommendation`, `model\_scope` y `routing\_reason` para que el usuario entienda por qué se generó cada predicción.

Resumen

Inferencia individual

Batch CFO

Evaluación

KPIs

Feedback

Model Registry

Feedback

Tienda

2 — Adygea TC "Mega"

Producto

30 — 007: COORDINATES "SKAYFOLL"

O escribe shop_id

Ej. 2

O escribe item_id

Ej. 2

Tipo de observación

Predicción muy alta

Nombre del analista

Comentario

Guardar feedback en RDS

Feedback capturado

	feedback_id	shop_id	item_id	issue_type	comment	analyst_name	created_at
0	11	36	30	Otro	Los productos de esta tienda no suelen venderse mucho	Hector Peralta	2026-05-02 19:33:39
1	10	42	13241	Predicción muy baja	Este producto está siendo muy solicitado y se está subestimando su demanda	Hector Peralta	2026-05-02 15:54:22

Figura 14: Captura de feedback de negocio.

100 productos más subestimados

Predicción menor que el real. Riesgo principal: subabasto.

	shop_id	shop_name	item_id	item_name	item_category_id	item_category_name	pr
10	28	Moscow shopping center "MEGA Teply Stan" II of	10202	The Witcher 3: Wild Hunt - Supplement "Stone Heart" (boot code, without disc) [PS4,	20	Games - PS4	
11	37	Novosibirsk TC "Mega"	10202	The Witcher 3: Wild Hunt - Supplement "Stone Heart" (boot code, without disc) [PS4,	20	Games - PS4	
12	37	Novosibirsk TC "Mega"	10201	Ведьмак 3: Дикая Охота - Дополнение "Каменные Сердца" (код загрузки, без ди	28	Games PC - Additional publications	
13	42	St. Petersburg TK "Nevsky Center"	7223	Uncharted: Nathan Drake. Collection [PS4, Russian version]	20	Games - PS4	
14	21	Moscow MTRTS "Afi Mall"	7224	Uncharted: Nathan Drake. Collection. Special edition [PS4, Russian version]	20	Games - PS4	
15	18	Krasnoyarsk SC "June"	10201	Ведьмак 3: Дикая Охота - Дополнение "Каменные Сердца" (код загрузки, без ди	28	Games PC - Additional publications	
16	39	RostovNaDonu TRC "Megacenter Horizon"	16629	LEFT (BD)	37	Movies - Blu-Ray	
17	22	Moscow Shop C21	1583	Assassin's Creed: Синдикат. Специальное издание [PS4, русская версия]	20	Games - PS4	
18	25	Moscow TRC "Atrium"	11704	ЗЕМЛЯ БУДУЩЕГО (BD)	37	Movies - Blu-Ray	
19	48	Tomsk SEC "Emerald City"	10202	The Witcher 3: Wild Hunt - Supplement "Stone Heart" (boot code, without disc) [PS4,	20	Games - PS4	
20	48	Tomsk SEC "Emerald City"	10201	Ведьмак 3: Дикая Охота - Дополнение "Каменные Сердца" (код загрузки, без ди	28	Games PC - Additional publications	

Registros sugeridos para revisión

	shop_id	shop_name	item_id	item_name	item_category_id	item_category_name	pr
0	24	Moscow TK "Budenovsky" (pav.K7)	1583	Assassin's Creed: Синдикат. Специальное издание [PS4, русская версия]	20	Games - PS4	
1	28	Moscow shopping center "MEGA Teply Stan" II of	1583	Assassin's Creed: Синдикат. Специальное издание [PS4, русская версия]	20	Games - PS4	
2	25	Moscow TRC "Atrium"	12612	КНЯЗЬ Предвестник (фирм.)	55	Music - CD of local production	
3	26	Moscow shopping mall "area" (Belyaev)	7223	Uncharted: Nathan Drake. Collection [PS4, Russian version]	20	Games - PS4	
4	25	Moscow TRC "Atrium"	13241	Witcher comic fox kids	47	Books - Comics, Manga	
5	25	Moscow TRC "Atrium"	9927	Self / Less	40	Movie - DVD	

Figura 15: Listado de productos sugeridos para revisión.

10.8 Model Registry

Model Registry muestra el champion, métricas y corridas evaluadas. La vista conserva el LightGBM original de dos etapas como incumbent, además de Hurdle HGB, HGB Poisson, especialista recurrente, rolling mean, promedio histórico y naive.

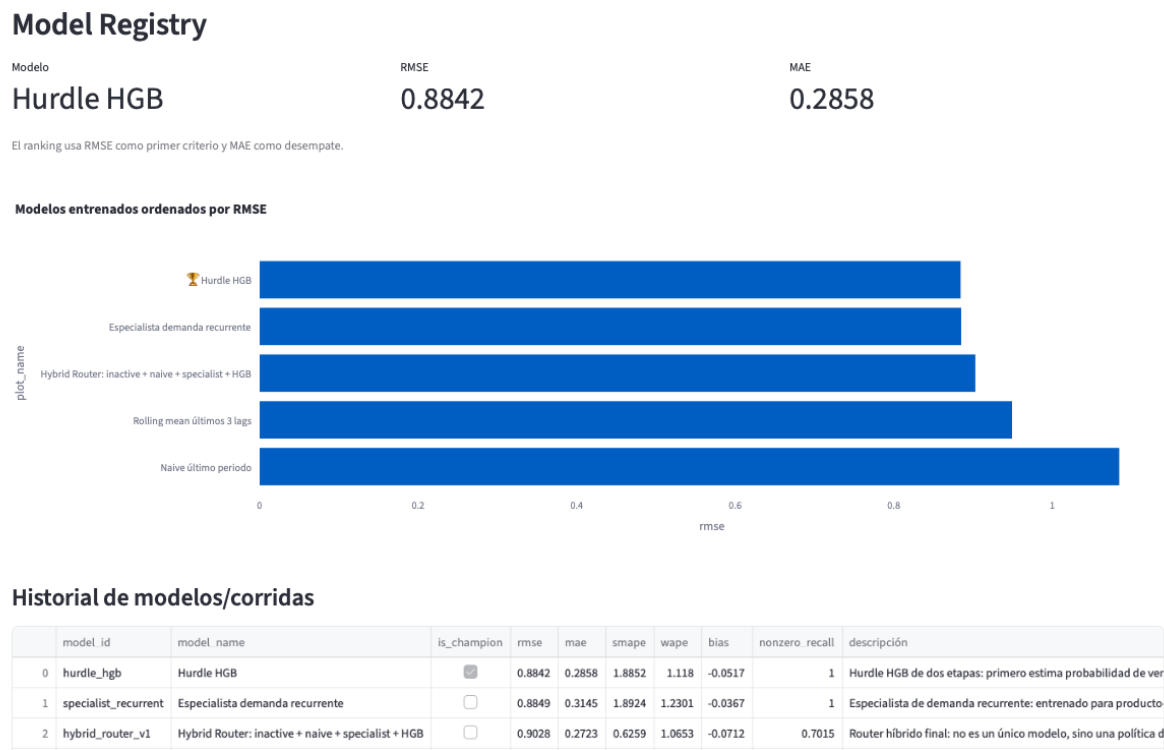


Figura 16: Vista Model Registry con historial de modelos y descripciones.

11 Evidencias de AWS

Estas capturas deben tomarse desde la consola de AWS y guardarse con los nombres indicados. La intención es demostrar que la app no corre localmente y que los servicios requeridos por la rúbrica están desplegados.

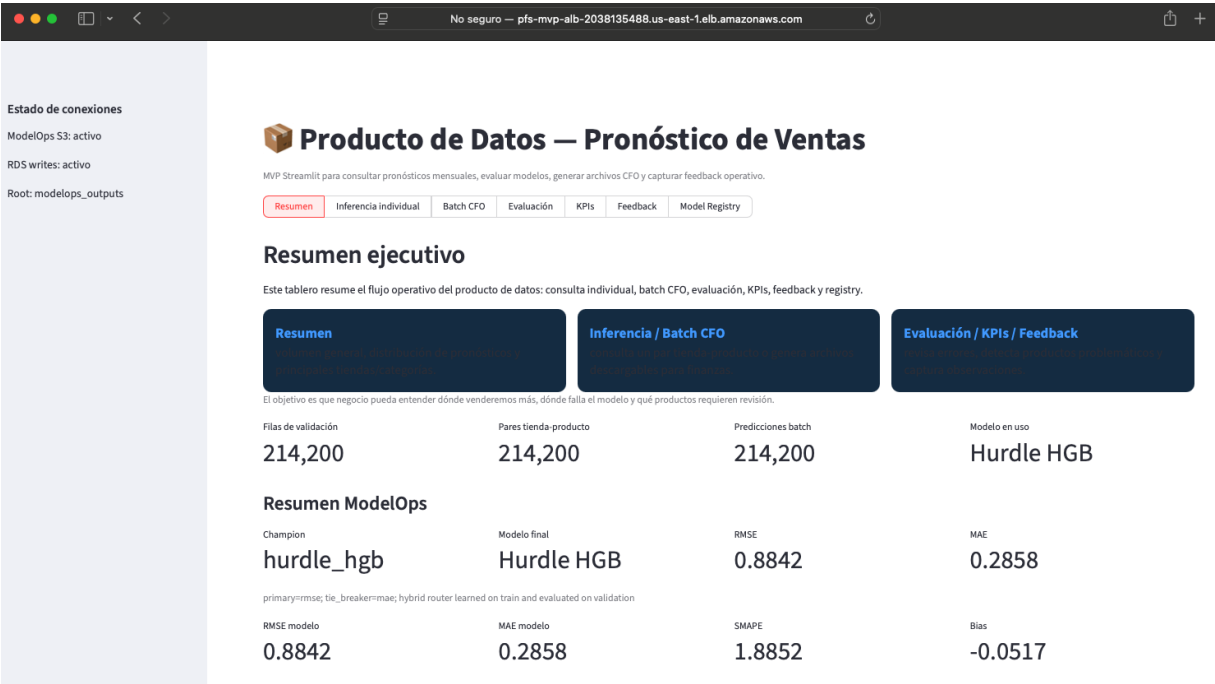


Figura 17: Endpoint público de la aplicación Streamlit abierto en navegador.

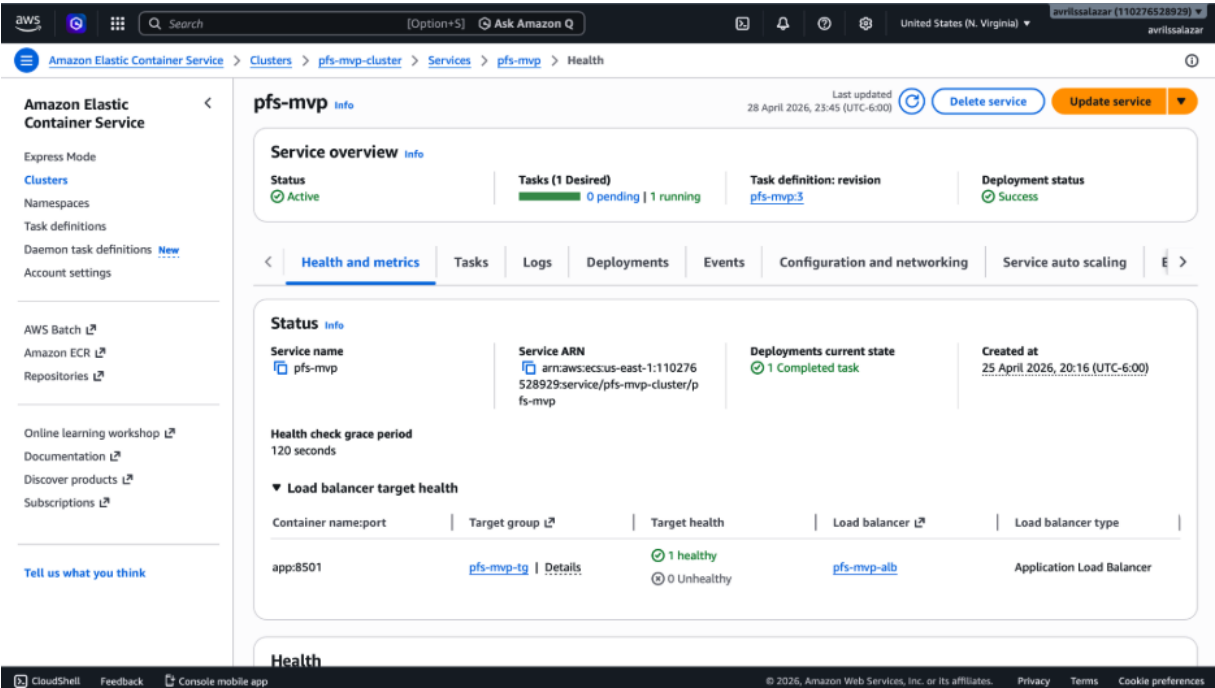


Figura 18: Servicio ECS Fargate con la tarea corriendo.

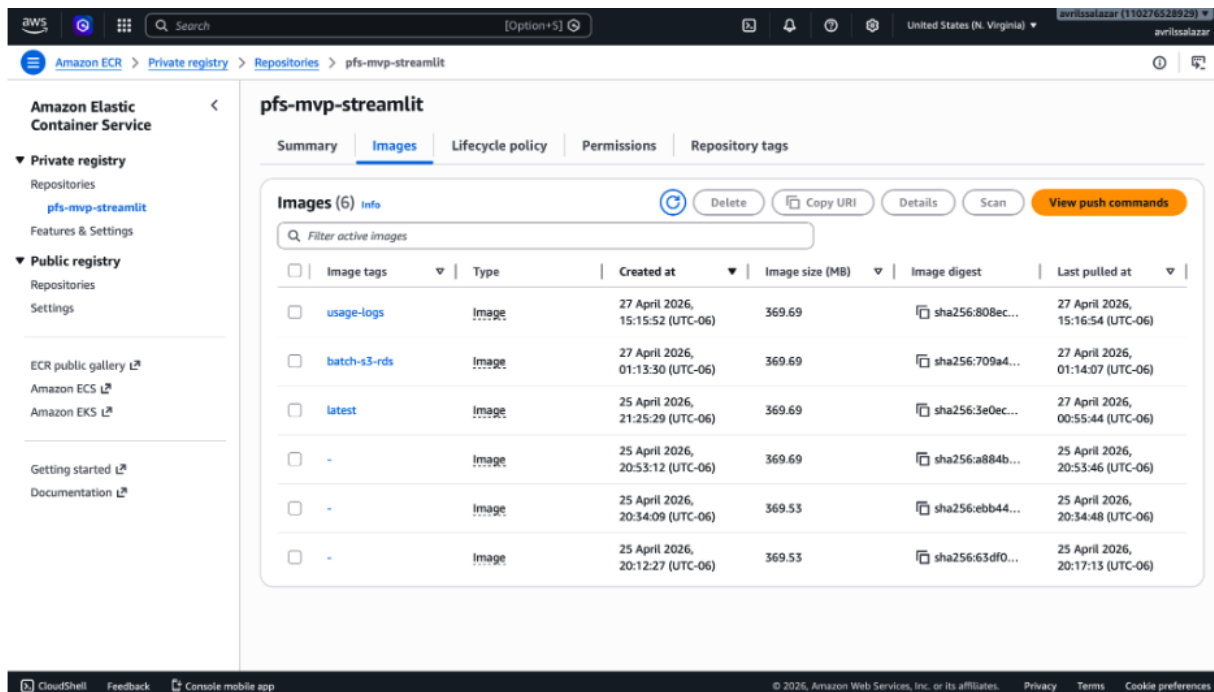


Figura 19: Repositorio ECR con la imagen Docker publicada.

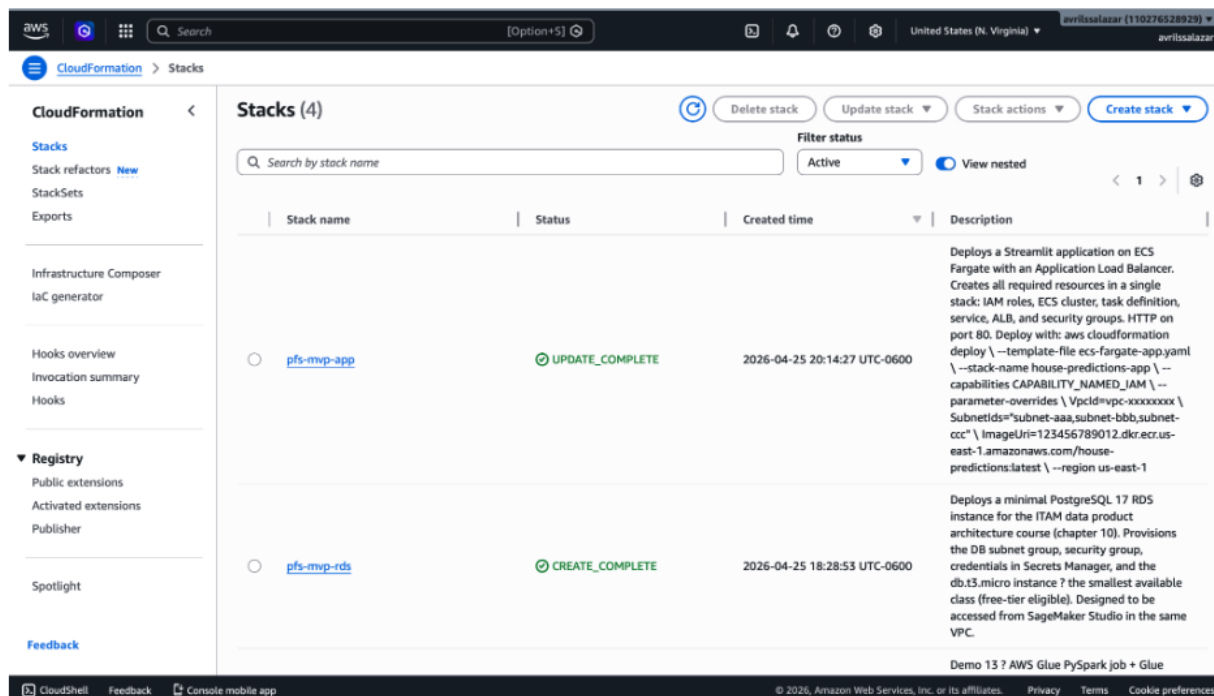


Figura 20: Stacks de CloudFormation en estado completo.

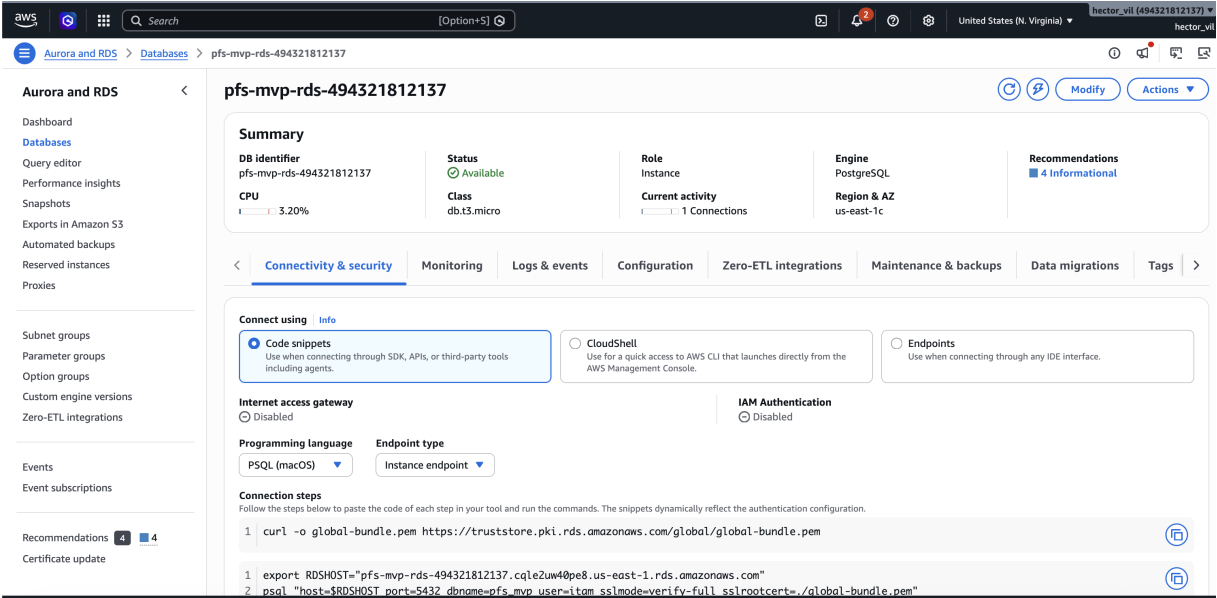


Figura 21: Instancia RDS PostgreSQL disponible.

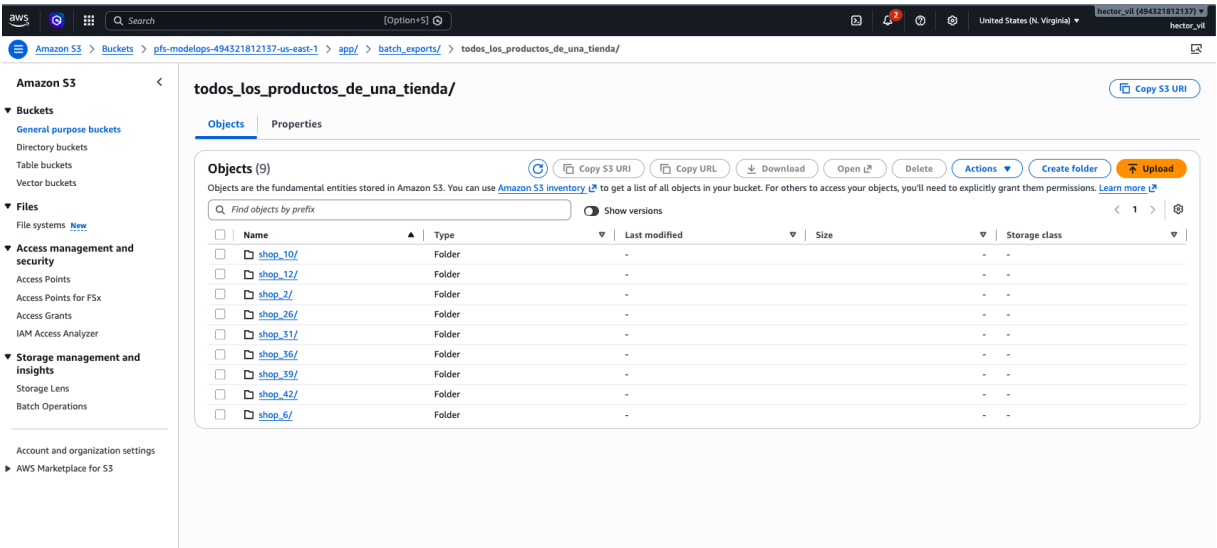


Figura 22: Archivos CFO y predicciones cargadas guardados en S3.

Figura 23: Tablas registradas en AWS Glue Data Catalog.

Figura 24: CloudWatch Logs del servicio ECS/Streamlit.

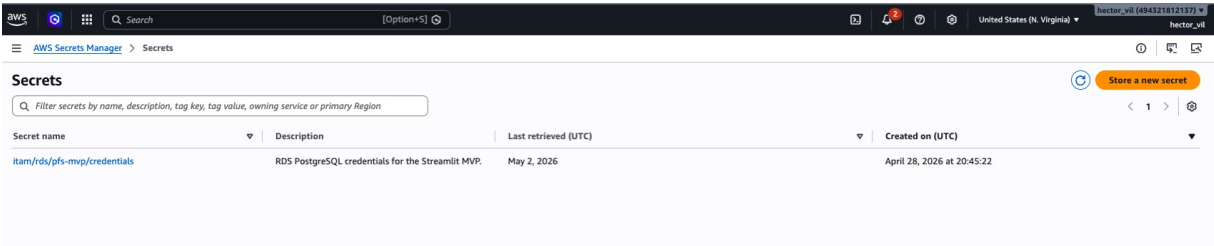


Figura 25: Secreto de RDS en AWS Secrets Manager.

12 Costos y operación

La estimación considera la versión estabilizada para demo: una tarea ECS Fargate corriendo 24/7 con 2 vCPU y 4 GB de memoria, un Application Load Balancer, RDS PostgreSQL pequeño, almacenamiento S3/ECR bajo, CloudWatch Logs, Glue Data Catalog y un secreto de Secrets Manager. Se usaron 730 horas mensuales.

Los precios de referencia se tomaron de páginas públicas de AWS: Fargate cobra por vCPU y memoria aprovisionada mientras la tarea está corriendo; el ALB cobra una tarifa por hora y por LCU; RDS cobra horas de instancia y almacenamiento; Secrets Manager cobra por secreto; Glue Data Catalog tiene un primer millón de objetos y accesos sin costo; ECR cobra almacenamiento de imágenes; y CloudWatch Logs cobra por ingesta de logs. Las fuentes se listan al final del reporte.

Cuadro 11: Estimación mensual de costos del MVP en AWS, región us-east-1.

Componente	Supuesto	USD/mes
ECS Fargate	2 vCPU, 4 GB, 730 h	72.08
Application Load Balancer	730 h base + 1 LCU promedio	22.27
RDS PostgreSQL	db.t3.micro 730 h	13.14
RDS Storage	20 GB gp2/gp3 aprox.	2.30
Secrets Manager	1 secreto	0.40
S3 Standard	10 GB de archivos/modelos/exports	0.23
ECR	5 GB de imágenes Docker	0.50
CloudWatch Logs	2 GB de logs ingeridos	1.00
Glue Data Catalog	Menos de 1M objetos/accesos	0.00
Total estimado versión estable		113. - 116 USD

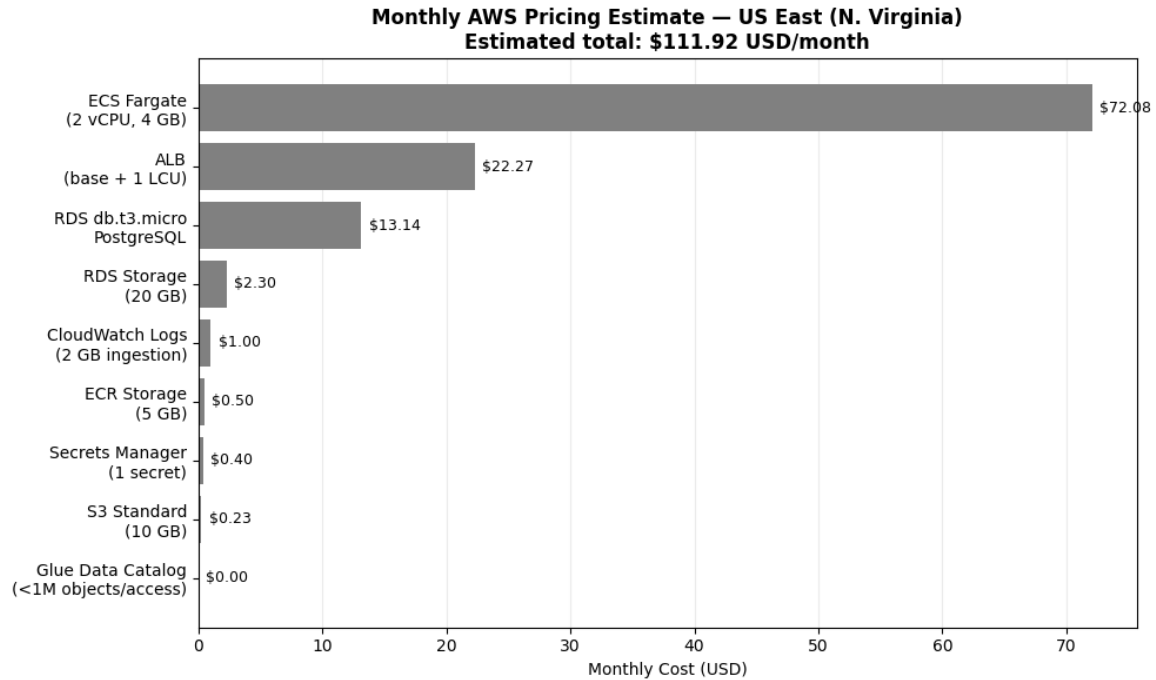


Figura 26: Estimación mensual de costos en AWS para la arquitectura estabilizada del MVP.

El costo de Fargate se calculó con las tarifas publicadas para Linux/x86 en US East: 0.000011244 USD por vCPU-segundo y 0.000001235 USD por GB-segundo. Al convertir a horas, esto equivale aproximadamente a 0.0404784 USD por vCPU-hora y 0.004446 USD por GB-hora. Para 2 vCPU y 4 GB durante 730 horas:

$$(2 \times 0.0404784 + 4 \times 0.004446) \times 730 = 72.08 \text{ USD/mes}$$

Si después de la demo se baja la tarea a 1 vCPU y 2 GB, el costo de Fargate baja a 36.04 USD/mes y el total mensual queda alrededor de 77 USD. Si se apaga el servicio fuera de horario, el costo baja todavía más porque Fargate y ALB son los componentes que más pesan cuando la app está encendida.

Desde el punto de vista de operación, los recursos que conviene apagar cuando el MVP no esté en uso son principalmente ECS *Fargate*, el *Application Load Balancer* y RDS, ya que son los componentes que generan costo por mantenerse activos. Para detener la aplicación, se puede bajar el número de tareas deseadas del servicio ECS a cero o eliminar el *stack pfs-mvp-app* desde *CloudFormation*. Para la base de datos, se puede detener o eliminar RDS mediante el *stack pfs-mvp-rds*, dependiendo de si se quiere conservar o no la información operacional.

Los recursos de S3 y ECR pueden mantenerse si se desea conservar los archivos generados, los artefactos del modelo y las imágenes *Docker* del MVP. En producción, también se recomienda configurar alertas de facturación, revisar el uso real del *Load Balancer* y considerar instancias reservadas de RDS si la solución se mantiene activa por más tiempo.

13 Operación, estabilidad y seguridad

Durante las pruebas apareció un problema de navegación pesada y errores 502. La causa no fue una falla del modelo, sino que Streamlit recalculaba demasiadas vistas al navegar. Se cambió la navegación para que renderizara una sección a la vez. También se aumentaron recursos de Fargate y se ajustó el ALB para mejorar estabilidad de sesiones largas.

La app usa CloudWatch Logs para depurar despliegues, errores de permisos S3 y reinicios. En S3 se separaron dos flujos: CFO exports y predicciones por archivo cargado. Esto permite revisar qué archivos se generaron para Finanzas y qué archivos fueron corridas ad hoc.

En seguridad, las credenciales de RDS se manejan con Secrets Manager y no se guardan en el código. Los permisos S3 del task role se limitaron por prefijo: `app/batch\_exports/` para CFO y `app/batch\_uploads/` para predicciones cargadas. En un ambiente productivo se agregaría autenticación de usuarios, autorización por rol y cifrado/rotación formal de secretos.

14 Cobertura de la rúbrica

Cuadro 12: Cobertura de puntos solicitados en el examen.

Punto solicitado	Cobertura en el proyecto
App Streamlit pública en AWS	Desplegada en ECS Fargate detrás de ALB, con URL pública.
Inferencia batch	Batch CFO para tienda/categoría/catálogo; batch por archivo cargado con features completas, estilo test y cold-start.
Diagrama draw.io	Debe incluirse como <code>figures/fig_01_arquitectura_general.png</code> y archivo <code>.drawio</code> en repo.
Servicios AWS mínimos	S3, ECS/Fargate, ECR, Glue, RDS, CloudFormation, Secrets Manager y CloudWatch aparecen en arquitectura. SageMaker Batch Transform se justifica como no usado en MVP.
Calidad visual y consumo de información	Vistas de Resumen, Evaluación, KPIs, Feedback, Batch CFO y Registry con tablas filtrables y explicaciones de columnas.
Calidad de predicciones	Comparación contra naive, model registry, curvas por bucket, políticas para productos inactivos y demanda recurrente.
RDS / persistencia	Feedback, historial CFO y eventos operativos persisten fuera del contenedor.

Punto solicitado	Cobertura en el proyecto
Reporte y evidencias	Este documento incluye placeholders de capturas y secciones de costos, operación, limitaciones y uso de IA.

15 Limitaciones y próximos pasos

El MVP cumple su función de demostrar el producto de datos completo, pero todavía hay aspectos que conviene resolver antes de producción.

- **Autenticación.** La demo usa URL pública. En producción se necesita login corporativo y permisos por rol.
- **Automatización de ModelOps.** El flujo de features, entrenamiento, evaluación y publicación debería ejecutarse de forma programada cuando lleguen nuevos datos.
- **Monitoreo de drift.** La app muestra métricas de validación, pero en producción habría que monitorear drift de datos y desempeño post-despliegue.
- **Notificaciones.** El flujo CFO podría enviar correo o Slack cuando un archivo esté listo.
- **SageMaker Batch Transform.** No se usó en el MVP por costo/simplicidad, pero sería útil si las corridas batch crecen o si se quiere desacoplar totalmente la inferencia de Streamlit.
- **Mejor manejo de productos nuevos.** Para pares sin historial se usa cold-start conservador. En producción habría que enriquecer con catálogos de negocio, jerarquías de producto y señales de lanzamiento.

16 Uso de herramientas de IA

Durante el proyecto se usó ChatGPT como apoyo para ordenar ideas, revisar errores, proponer fragmentos de código, estructurar documentación y depurar mensajes de AWS, Docker, Streamlit, ECS, RDS y S3. También ayudó a convertir decisiones técnicas en explicaciones más claras para el reporte.

Las decisiones finales de arquitectura, la ejecución de comandos, las pruebas locales, las capturas, la validación en AWS y la integración final fueron revisadas y ejecutadas por el equipo. La herramienta se usó como apoyo de ingeniería y documentación, no como sustituto del trabajo técnico.

17 Bibliografía y recursos consultados

Referencias

- [1] Rožanec, J. M., Petelin, G., Costa, J., Bertalanič, B., Cerar, G., Guček, M., Papa, G. y Mladenčić, D. (2023). *Dealing with zero-inflated data: achieving SOTA with a two-fold machine learning approach*. arXiv:2310.08088. Disponible en: <https://arxiv.org/pdf/2310.08088>
- [2] OpenAI. (2026). *ChatGPT, versión GPT-5.4*. Modelo de lenguaje utilizado como apoyo para estructurar el reporte, revisar redacción, documentar decisiones técnicas y depurar fragmentos de código. Disponible en: <https://chat.openai.com/>
- [3] Amazon Web Services. (2026). *AWS Fargate Pricing*. Disponible en: <https://aws.amazon.com/fargate/pricing/>
- [4] Amazon Web Services. (2026). *Elastic Load Balancing Pricing*. Disponible en: <https://aws.amazon.com/elasticloadbalancing/pricing/>
- [5] Amazon Web Services. (2026). *Amazon RDS for PostgreSQL Pricing*. Disponible en: <https://aws.amazon.com/rds/postgresql/pricing/>
- [6] Amazon Web Services. (2026). *Amazon S3 Pricing*. Disponible en: <https://aws.amazon.com/s3/pricing/>
- [7] Amazon Web Services. (2026). *Amazon ECR Pricing*. Disponible en: <https://aws.amazon.com/ecr/pricing/>
- [8] Amazon Web Services. (2026). *Amazon CloudWatch Pricing*. Disponible en: <https://aws.amazon.com/cloudwatch/pricing/>
- [9] Amazon Web Services. (2026). *AWS Secrets Manager Pricing*. Disponible en: <https://aws.amazon.com/secrets-manager/pricing/>
- [10] Amazon Web Services. (2026). *AWS Glue Pricing*. Disponible en: <https://aws.amazon.com/glue/pricing/>